

Phonebook Java Backend — Knowledge Bytes

Overall backend architecture

Byte 1: Overall backend architecture and request lifecycle

Builds on: None — starting point

In plain terms: Think of the backend as a restaurant kitchen. Controllers take orders at the counter (HTTP), services do the skilled prep (auth, hashing, IDs), repositories fetch ingredients from the pantry (MongoDB), and DTOs are the plates food is served on (JSON).

The code:

```
Browser → Nginx :80 (/api/*) → Spring Boot :8000 → Controller → Service →  
Repository/MongoTemplate → MongoDB 7  
Collections: users, contacts, tags, sessions, counters
```

What's happening: A request enters a Controller method, which first calls `AuthService.currentUser()` to learn who is calling (401 if unknown). Business rules run through Validators, IDs come from `SequenceService`, passwords/tokens go through `SecurityService`, and data access happens either through a Repository one-liner or a hand-built `MongoTemplate` query. Responses leave as DTO records serialized to `snake_case` JSON; failures leave as `{detail}` via `ApiException`. MongoDB holds five collections: users, contacts, tags, sessions, and counters (for numeric IDs).

Why it matters: Every later byte is one station in this kitchen. When lost, ask: is this the counter, the prep cook, the pantry door, or the plate?

backend/src/main/java/com/phonebook/PhonebookApplication.java

Byte 2: PhonebookApplication — entry point

Builds on: Byte 1

In plain terms: Every Java program starts in a `main()` method. Here that method just hands control to Spring Boot, which builds the whole kitchen and opens the doors.

The code:

```
package com.phonebook;  
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PhonebookApplication {
    public static void main(String[] args) {
        SpringApplication.run(PhonebookApplication.class, args);
    }
}
```

What's happening: Package `com.phonebook` is the scan root: Spring searches it and below for components. The `@SpringBootApplication` annotation bundles `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`, so the web server, MongoDB wiring, controllers, services, and the two `ApplicationRunner` beans all register themselves. `main()` takes CLI args (notably the word 'populate', used by `PopulateConfig`) and blocks serving HTTP until startup — including index creation — finishes.

Why it matters: Without this file nothing runs. It is also why every other file must live under `com.phonebook` to be discovered.

backend/src/main/resources/application.properties

Byte 3: Main config — port and MongoDB address

Builds on: Byte 2

In plain terms: This is the settings panel: which port to listen on and where MongoDB lives. Docker and local runs share the file; only an environment variable changes.

The code:

```
server.port=8000
spring.application.name=phonebook-api
spring.data.mongodb.uri=${MONGODB_URI:mongodb://localhost:27017/phonebook_db}
```

What's happening: `server.port=8000` is why Nginx proxies to `backend:8000`. `spring.application.name` labels logs. `spring.data.mongodb.uri` reads the `MONGODB_URI` environment variable (Docker Compose sets `mongodb://phonebook:...@db:27017/phonebook_db`) and falls back after the colon to `localhost` for plain local runs. No code mentions hostnames — this line is the only place.

Why it matters: Change databases, move hosts, or remap ports here, not in Java. Misconfigure this and every repository call fails at startup.

Byte 4: Main config — JSON shape and logging

Builds on: Byte 3

In plain terms: Three Jackson settings decide what JSON looks like on the wire; one logging line keeps MongoDB chatter visible.

The code:

```
spring.jackson.property-naming-strategy=SNAKE_CASE
spring.jackson.mapper.accept-case-insensitive-properties=true
spring.jackson.serialization.write-dates-as-timestamps=false
logging.level.org.springframework.data.mongodb.core.MongoTemplate=INFO
```

What's happening: SNAKE_CASE serializes phoneNumber as phone_number automatically (explicit @JsonProperty annotations in DTOs pin the important ones regardless). accept-case-insensitive-properties lets the API also accept phoneNumber, which is why controllers read both spellings. write-dates-as-timestamps=false renders Instants as ISO strings like 2026-09-16T07:37:26.876Z. The logging line surfaces MongoTemplate queries at INFO.

Why it matters: Frontend field names (tag_ids, total_pages, created_at) and tolerant input casing both come from these four lines.

backend/pom.xml

Byte 5: Build recipe — platform and core dependencies

Builds on: Byte 1

In plain terms: pom.xml is the shopping list Maven uses to download libraries and compile the project with Java 17 and Spring Boot 3.2.5.

The code:

```
<parent><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId><version>3.2.5</version></parent>
<properties><java.version>17</java.version></properties>
<groupId>com.phonebook</groupId><artifactId>phonebook-api</artifactId>
<version>1.0.0</version><packaging>jar</packaging>
<dependencies>
<artifactId>spring-boot-starter-web</artifactId> <!-- Tomcat + MVC + JSON -->
<artifactId>spring-boot-starter-data-mongodb</artifactId>
<artifactId>spring-boot-starter-validation</artifactId>
```

What's happening: The parent pins compatible versions for all Spring artifacts so dependency versions rarely need stating. starter-web brings the embedded Tomcat server, @RestController support, and JSON conversion. starter-data-mongodb brings MongoRepository, MongoTemplate, and the @Document/@Indexed annotations. starter-validation is on the classpath though most validation here is hand-written in Validators.

Why it matters: Every import starting with org.springframework exists because of this file. Bump the parent version and the whole framework moves.

Byte 6: Build recipe — CSV, crypto, and test libraries

Builds on: Byte 5

In plain terms: Three more dependencies cover the remaining superpowers: parsing CSV uploads, hashing passwords with scrypt, and running tests.

The code:

```
<dependency><groupId>org.apache.commons</groupId>
  <artifactId>commons-csv</artifactId><version>1.11.0</version></dependency>
<dependency><groupId>org.bouncycastle</groupId>
  <artifactId>bcprov-jdk18on</artifactId><version>1.78.1</version></dependency>
<dependency><groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>
<build><plugins>
  <artifactId>spring-boot-maven-plugin</artifactId> <!-- builds runnable jar -->
</plugins></build>
```

What's happening: commons-csv powers CSVParser/CSVFormat in the import endpoint. Bouncy Castle's bcprov supplies SCrypt.generate() used by SecurityService. starter-test (JUnit 5, MockMvc, AssertJ, Mockito) is scope=test, so it ships in no production jar. The Spring Boot Maven plugin repackages target/phonebook-api-1.0.0.jar with an embedded server — the exact file the Dockerfile copies and runs.

Why it matters: Remove commons-csv and import breaks; remove Bouncy Castle and nobody can log in. The Dockerfile's jar path must match artifactId/version here.

backend/src/main/java/com/phonebook/model/Contact.java

Byte 7: Contact model — fields

Builds on: Bytes 1, 6

In plain terms: A Contact is one phonebook row: its owner, name, phone, optional email/address, creation time, and the tag IDs stuck to it.

The code:

```
package com.phonebook.model;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
```

```

@Document(collection = "contacts")
public class Contact {
    @Id private Long id;                // numeric id from SequenceService
    private Long userId;                // owner — every query filters on this
    private String name;
    private String phoneNumber;         // globally unique (DB index)
    private String email;               // optional, globally unique when present
    private String address;             // optional
    private Instant createdAt;
    private List<Long> tagIds = new ArrayList<>(); // embedded tag references
}

```

What's happening: @Document binds the class to the contacts collection; @Id marks the Mongo _id (a Long here, not the default ObjectId, because SequenceService assigns it). userId is the isolation key — repository methods like findByIdAndUserId always include it. phoneNumber/email uniqueness is global across users, enforced by the indexes in Byte 72. tagIds embeds references instead of a join table, which is why tag deletion must scrub contacts.

Why it matters: This is the most-touched class in the backend. Every contact endpoint reads or writes these eight fields.

Byte 8: Contact model — accessors

Builds on: Byte 7

In plain terms: Plain getters and setters expose each field, with one defensive setter that refuses null tag lists.

The code:

```

public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public Long getUserId() { return userId; }
public void setUserId(Long userId) { this.userId = userId; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getPhoneNumber() { return phoneNumber; }
public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getAddress() { return address; }
public void setAddress(String address) { this.address = address; }
public Instant getCreatedAt() { return createdAt; }
public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
public List<Long> getTagIds() { return tagIds; }
public void setTagIds(List<Long> tagIds) { this.tagIds = tagIds != null ? tagIds : new ArrayList<>(); }

```

What's happening: Each getter/setter is a trivial field proxy — Spring and Jackson use them for persistence and JSON. The only logic is in `setTagIds`: a null argument becomes an empty list, so callers (constructors, import, tag-scrubbing loop) can never leave `tagIds` null and cause `NullPointerExceptions` downstream.

Why it matters: Boring but load-bearing: the null-guard is why code like `c.getTagIds().removeIf(...)` is safe everywhere.

backend/src/main/java/com/phonebook/model/User.java

Byte 9: User model — fields and constructor

Builds on: Bytes 1, 6

In plain terms: A User is an account: login names plus a scrambled password hash. The real password never touches the database.

The code:

```
package com.phonebook.model;
import java.time.Instant;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "users")
public class User {
    @Id private Long id;
    @Indexed(unique = true) private String username;
    @Indexed(unique = true) private String email;
    private String passwordHash; // "scrypt$salt$digest" – never plaintext
    private Instant createdAt;
    public User() {}
}
```

What's happening: `@Indexed(unique = true)` on `username` and `email` makes MongoDB itself reject duplicates — the backstop behind the application's friendly 409 checks, and the reason the tag/contact unique races are also safe. The explicit public no-arg constructor exists because Spring and Jackson instantiate models reflectively; without it, deserialization fails.

Why it matters: Registration writes one of these; login reads one. The `passwordHash` format is produced and verified by `SecurityService`.

Byte 10: User model — accessors

Builds on: Byte 9

In plain terms: Straightforward getters and setters for all five fields, no logic.

The code:

```
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getPasswordHash() { return passwordHash; }
public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
public Instant getCreatedAt() { return createdAt; }
public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
```

What's happening: Pure boilerplate: each method reads or writes exactly one field. Controllers call setUsername/setEmail/setPasswordHash/setCreatedAt during registration and getId/getUsername/getEmail when building responses and sessions.

Why it matters: Nothing can break here unless a field is renamed without updating its accessor — then JSON and queries silently miss it.

backend/src/main/java/com/phonebook/model/Tag.java

Byte 11: Tag model — fields

Builds on: Bytes 1, 6

In plain terms: A Tag is a label like Work. Each user owns a separate tag namespace, and matching ignores capitalization.

The code:

```
package com.phonebook.model;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "tags")
public class Tag {
    @Id private Long id;
    private Long userId;      // owner — tags are per-user
    private String name;      // display form, e.g. "Work"
    private String lowerName; // lookup key, e.g. "work"
```

What's happening: userId scopes every tag operation the same way contacts are scoped. name keeps the user's capitalization for display while lowerName is the normalized key used in uniqueness checks and the compound unique index (userId + lowerName).

Why it matters: Two users can both own Work; one user cannot own both Work and work. That rule lives in this pair of fields.

Byte 12: Tag model — name sync and accessors

Builds on: Byte 11

In plain terms: setName keeps the lookup key in sync automatically; the remaining accessors are plain.

The code:

```
public void setName(String name) {
    this.name = name;
    this.lowerName = name != null ? name.toLowerCase() : null;
}
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public Long getUserId() { return userId; }
public void setUserId(Long userId) { this.userId = userId; }
public String getName() { return name; }
public String getLowerName() { return lowerName; }
public void setLowerName(String lowerName) { this.lowerName = lowerName; }
```

What's happening: Deriving lowerName inside setName (rather than trusting callers) guarantees the invariant survives renames: TagController.update only calls setName and uniqueness still holds. The null branch prevents NullPointerException on defensive calls. setLowerName exists for the persistence layer but application code should prefer setName.

Why it matters: If lowerName ever drifted from name, duplicate detection would silently fail — this setter is the single point that prevents that.

backend/src/main/java/com/phonebook/model/AuthSession.java

Byte 13: AuthSession model — all fields and accessors

Builds on: Bytes 1, 6

In plain terms: One row per logged-in browser: a fingerprint of its secret, whose it is, and when it dies.

The code:

```
package com.phonebook.model;
import java.time.Instant;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;
```

```

@Document(collection = "sessions")
public class AuthSession {
    @Id private String id;    // Mongo default key
    @Indexed(unique = true) private String tokenHash; // SHA-256 hex of the cookie
    token
    private Long userId;
    private Instant expiresAt;    // now + 7 days at issue
    private Instant createdAt;
    // plain
    getId/setId/getTokenHash/setTokenHash/getUserId/setUserId/getExpiresAt/setExpiresAt/get
    CreatedAt/setCreatedAt
}

```

What's happening: The @Id is a String (Mongo's default key type) while tokenHash carries the unique lookup index — sessions are found by hash, never by id. Only the hash is persisted, so a database leak exposes no live cookies (see SecurityService). expiresAt is checked in AuthService.currentUser on every request; there is no MongoDB TTL index, so expired rows linger until overwritten. All accessors are trivial field proxies.

Why it matters: Login inserts one row, each authenticated request reads it, logout deletes it. Lose this collection and every session ends.

backend/src/main/java/com/phonebook/model/Counter.java

Byte 14: Counter model — all fields and accessors

Builds on: Bytes 1, 6

In plain terms: One tiny row per ID sequence (users, contacts, tags) remembering the last number handed out.

The code:

```

package com.phonebook.model;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "counters")
public class Counter {
    @Id private String id; // sequence name, e.g. "contacts"
    private long seq;      // last issued value
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public long getSeq() { return seq; }
    public void setSeq(long seq) { this.seq = seq; }
}

```

What's happening: The `@Id` doubles as the sequence name, so the collection holds exactly three documents. `seq` is a primitive long (never null). `SequenceService` atomically increments it with `$inc` — this class is just the shape of that document.

Why it matters: Delete a counter row and IDs restart from 1, colliding with existing data. It is small but must never be hand-edited.

backend/src/main/java/com/phonebook/dto/Dtos.java

Byte 15: DTOs — auth, user, and tag records

Builds on: Bytes 6, 7

In plain terms: DTOs are the exact JSON shapes crossing the wire — separate classes so internal fields like `passwordHash` can never leak into responses.

The code:

```
package com.phonebook.dto;
import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;
import java.time.Instant;
import java.util.List;

public final class Dtos {
    private Dtos() {} // utility holder – never instantiated
    public record RegisterRequest(String username, String email, String password) {}
    public record LoginRequest(String identifier, String password) {}
    public record UserResponse(long id, String username, String email, Instant
createdAt) {}
    public record TagRequest(String name) {}
    public record TagResponse(long id, String name) {}
}
```

What's happening: Java records are immutable carriers whose components serialize directly to JSON. `LoginRequest` takes one identifier (username OR email) plus password. `UserResponse` deliberately omits the password hash. UNUSED CODE: `RegisterRequest`, `LoginRequest`, and `TagRequest` are declared but never referenced — every controller actually binds `@RequestBody Map<String, Object>` raw and extracts fields by hand, so these three records are dead contract documentation.

Why it matters: `TagResponse` and `UserResponse` are used constantly; the three Request records can be deleted without effect (or adopted to replace the Map binding).

Byte 16: DTOs — contact request and response

Builds on: Byte 15

In plain terms: Contact JSON uses snake_case on the wire while Java code stays camelCase; responses embed full tag objects, not bare IDs.

The code:

```
@JsonInclude(JsonInclude.Include.ALWAYS)
public record ContactRequest(
    String name,
    @JsonProperty("phone_number") String phoneNumber,
    String email,
    String address,
    @JsonProperty("tag_ids") List<Long> tagIds) {}

public record ContactResponse(
    long id,
    String name,
    @JsonProperty("phone_number") String phoneNumber,
    String email,
    String address,
    Instant createdAt,
    List<TagResponse> tags) {}
```

What's happening: @JsonProperty pins wire names regardless of the global SNAKE_CASE strategy. ContactResponse.tags holds resolved TagResponse objects (id+name, sorted) — the controllers translate stored tagIds into these via toResponse/toResponses. **UNUSED CODE:** ContactRequest is never bound by any endpoint (create/update read raw Maps and even accept camelCase aliases), so it documents intent but enforces nothing.

Why it matters: The contact form, cards, and detail panel all speak ContactResponse. If tags ever render as IDs instead of pills, the resolver — not this record — is the culprit.

Byte 17: DTOs — page envelope and message wrappers

Builds on: Bytes 12, 16

In plain terms: List results travel in a pagination envelope; success and failure messages share two tiny wrappers.

The code:

```
public record ContactPage(
    List<ContactResponse> items,
    int page,
    int limit,
    long total,
    @JsonProperty("total_pages") int totalPages) {}

public record ErrorResponse(String detail) {}
public record MessageResponse(String message) {}
```

What's happening: ContactPage carries the slice (items) plus the numbers the UI needs: current page, page size, grand total, and page count. ErrorResponse({detail: string}) is the body ApiException builds for every thrown error; MessageResponse({message}) wraps delete/logout confirmations. Both serialize to a single JSON key.

Why it matters: Numbered pagination, the Delete/Logout toasts, and every error banner in the frontend depend on these exact key names.

Byte 18: DTOs — import report and unused validation record

Builds on: Bytes 13, 17

In plain terms: A CSV import returns a detailed report card, and one leftover record documents a validation shape nobody uses.

The code:

```
public record ValidationError(String loc0, String loc1, String msg, String type) {}

public record ImportRowError(int row, String error) {}
public record ImportSummary(
    @JsonProperty("total_rows") int totalRows,
    int imported,
    @JsonProperty("skipped_duplicates") int skippedDuplicates,
    @JsonProperty("invalid_rows") int invalidRows,
    @JsonProperty("invalid_names") int invalidNames,
    @JsonProperty("invalid_phone_numbers") int invalidPhoneNumbers,
    @JsonProperty("invalid_emails") int invalidEmails,
    @JsonProperty("invalid_addresses") int invalidAddresses,
    @JsonProperty("row_errors") List<ImportRowError> rowErrors) {}
```

What's happening: ImportSummary breaks the outcome into totals plus per-field failure counters and a per-row error list — the frontend renders it almost verbatim as the import summary box. UNUSED CODE: ValidationError is never constructed (registration builds its 422 detail items as inline Maps), so it is aspirational typing only.

Why it matters: Import success/failure UX is this record. ValidationError could replace the inline maps in AuthController.register if someone wants typed errors.

<backend/src/main/java/com/phonebook/repo/UserRepository.java>

Byte 19: UserRepository — query contract

Builds on: Bytes 6, 7, 14

In plain terms: Five one-line declarations cover every user lookup; Spring generates the MongoDB queries from the method names.

The code:

```
package com.phonebook.repo;
import com.phonebook.model.User;
import java.util.Optional;
import org.springframework.data.mongodb.repository.MongoRepository;

public interface UserRepository extends MongoRepository<User, Long> {
    Optional<User> findByUsername(String username);    // DECLARED BUT UNUSED – no caller
    Optional<User> findByEmail(String email);        // DECLARED BUT UNUSED – no caller
    Optional<User> findByUsernameOrEmail(String username, String email); // login lookup
    boolean existsByUsername(String username);        // register duplicate check
    boolean existsByEmail(String email);              // register duplicate check
}
```

What's happening: `MongoRepository<User, Long>` donates `save/findById/findAll/delete/count/saveAll` for free; the interface needs no implementation class. `findByUsernameOrEmail` becomes a `username=? OR email=?` query — login calls it with (identifier, identifier). The `existsBy` pair backs the friendly 409s in registration. `findByUsername/findByEmail` compile into working queries but nothing calls them; they are harmless spare API surface.

Why it matters: Registration and login are thin wrappers over the last three methods. The first two can be deleted with zero behavioral change.

backend/src/main/java/com/phonebook/repo/ContactRepository.java

Byte 20: ContactRepository — scoped reads and counts

Builds on: Bytes 6, 14

In plain terms: User-scoped reads keep users isolated; the count methods serve pagination math and the populate runner.

The code:

```
public interface ContactRepository extends MongoRepository<Contact, Long> {
    Optional<Contact> findByIdAndUserId(Long id, Long userId); // auth gate: id AND owner
    List<Contact> findByUserIdOrderByNameAsc(Long userId);    // export + ownership listing
    long countByUserId(Long userId);
}
```

```
long count();    // REDUNDANT – MongoRepository already declares count()
```

What's happening: `findByIdAndUserId` is the authorization primitive: every get/update/delete resolves through it, turning foreign IDs into 404s. `findByUserIdOrderByNameAsc` sorts A→Z for export. `countByUserId` could serve per-user totals; `count()` (all contacts) is used by `PopulateConfig` and `AuthController`'s first-user check — but redeclaring it here is redundant since the parent interface already provides it.

Why it matters: Lose `findByIdAndUserId` and user isolation collapses. The `count()` line can be deleted safely.

Byte 21: `ContactRepository` — global uniqueness lookups

Builds on: Byte 20

In plain terms: Phone and email uniqueness is global (across all users), enforced by these four methods plus database indexes.

The code:

```
boolean existsByPhoneNumber(String phoneNumber); // create fast-path duplicate check
boolean existsByEmail(String email);             // create fast-path duplicate check
Optional<Contact> findByPhoneNumber(String phoneNumber); // update: fetch rival,
compare ids
Optional<Contact> findByEmail(String email);       // update: fetch rival,
compare ids
```

What's happening: Create uses `existsBy*` for a quick 409 before inserting; update uses `findBy*` to load the conflicting row and allow it when the rival is the contact itself (same id → not a conflict). A try/catch around `save()` converts index-race collisions into the same 409, so these checks are the friendly path and the unique index is the guarantee.

Why it matters: Duplicate-phone/email UX (409 with a clear message) comes from these four lines.

`backend/src/main/java/com/phonebook/repo/TagRepository.java`

Byte 22: `TagRepository` — listing, ownership, and name lookup

Builds on: Bytes 8, 14

In plain terms: Per-user tag queries: alphabetical listing, ownership-gated access, and case-insensitive name checks via `lowerName`.

The code:

```
public interface TagRepository extends MongoRepository<Tag, Long> {
    List<Tag> findByUserIdOrderByNameAsc(Long userId);           // tag list
    endpoint
    Optional<Tag> findByIdAndUserId(Long id, Long userId);       // rename/delete
    gate
    boolean existsByUserIdAndLowerName(Long userId, String lowerName); // create 409
    check
    Optional<Tag> findByUserIdAndLowerName(Long userId, String lowerName); // rename
    rival check
}
```

What's happening: `findByIdAndUserId` mirrors the contact auth gate (foreign tag → 404). The `lowerName` pair implements per-user case-insensitive uniqueness: create checks existence, rename loads the rival and compares IDs so a tag can keep its own name. All four are scoped by `userId` first — no cross-user leakage is expressible.

Why it matters: Tag listing, duplicate-name rejection, and safe renames all rest here.

Byte 23: TagRepository — batch and ownership-count queries

Builds on: Byte 22

In plain terms: Three bulk helpers: verify a contact's tags all belong to the caller, resolve IDs to tag objects, and one unused spare.

The code:

```
long countByUserIdAndIdIn(Long userId, List<Long> ids); // owned-count check
(ensureOwnedTags)
List<Tag> findByUserIdAndIdIn(Long userId, List<Long> ids); // DECLARED BUT UNUSED
List<Tag> findByIdIn(List<Long> ids);                       // resolve contact tagIds
→ objects
```

What's happening: `countByUserIdAndIdIn` returns how many of the requested IDs the user owns; `ContactController` requires `owned == requested`, else 400. `findByIdIn` batch-loads tag documents when building contact responses (one query for all contacts on the page, not N+1). `findByUserIdAndIdIn` is never called — the codebase chose the `count+findByIdIn` combination instead.

Why it matters: Assign-only-your-own-tags and the tag pills on contact cards both flow through the first and third methods.

backend/src/main/java/com/phonebook/repo/AuthSessionRepository.java

Byte 24: AuthSessionRepository — session lookup and revocation

Builds on: Bytes 9, 14

In plain terms: Two methods are the entire session store API: find by token fingerprint, delete by token fingerprint.

The code:

```
package com.phonebook.repo;
import com.phonebook.model.AuthSession;
import java.util.Optional;
import org.springframework.data.mongodb.repository.MongoRepository;

public interface AuthSessionRepository extends MongoRepository<AuthSession, String> {
    Optional<AuthSession> findByTokenHash(String tokenHash); // currentUser() lookup
    void deleteByTokenHash(String tokenHash);                // logout revocation
}
```

What's happening: The ID type is String (session documents use Mongo's default key). AuthService hashes the cookie token with SHA-256 and calls findByTokenHash on every authenticated request; revokeSession calls deleteByTokenHash on logout. No expiry query exists — staleness is checked in Java against expiresAt.

Why it matters: Login persistence and logout invalidation are exactly these two calls.

backend/src/main/java/com/phonebook/service/SequenceService.java

Byte 25: SequenceService — wiring and the atomic counter

Builds on: Bytes 10, 14

In plain terms: A tiny service that hands out 1, 2, 3... IDs for users, contacts, and tags without collisions, even under concurrency.

The code:

```
package com.phonebook.service;
import com.phonebook.model.Counter;
import org.springframework.data.mongodb.core.FindAndModifyOptions;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Query;
import org.springframework.data.mongodb.core.query.Update;
import org.springframework.stereotype.Service;
```

```

@Service
public class SequenceService {
    private final MongoTemplate mongoTemplate;
    public SequenceService(MongoTemplate mongoTemplate) {
        this.mongoTemplate = mongoTemplate;
    }

    public long next(String name) {
        Query query = new Query(Criteria.where("_id").is(name));
        Update update = new Update().inc("seq", 1);
        FindAndModifyOptions options = new
FindAndModifyOptions().returnNew(true).upsert(true);
        Counter counter = mongoTemplate.findAndModify(query, update, options,
Counter.class);
        return counter != null ? counter.getSeq() : 1L;
    }
}

```

What's happening: @Service registers it for dependency injection; constructor injection of MongoTemplate is the project's standard wiring style. next(name) runs findAndModify — MongoDB executes match-plus-\$inc atomically, so two simultaneous creates cannot receive the same number. upsert(true) creates the counter row on first use; returnNew(true) returns the post-increment value. The null fallback (1L) is defensive and unreachable in practice. Parameters: name is the sequence ('users'/'contacts'/'tags'). Returns the fresh ID.

Why it matters: Every created entity's numeric ID comes from here. Replace it with max+1 logic and concurrent creates would collide.

backend/src/main/java/com/phonebook/service/SecurityService.java

Byte 26: SecurityService — scrypt password hashing

Builds on: Bytes 7, 9

In plain terms: Passwords are never stored. Each is stretched through scrypt with a fresh random salt; only salt plus digest are kept.

The code:

```

@Service
public class SecurityService {
    private static final int SCRYPT_N = 16384;
    private static final int SCRYPT_R = 8;
    private static final int SCRYPT_P = 1;
    private static final int DK_LEN = 64;
    private final SecureRandom random = new SecureRandom();
}

```

```

public String hashPassword(String password) {
    byte[] salt = new byte[16];
    random.nextBytes(salt);
    byte[] digest = derive(password, salt);
    return "scrypt$" + base64UrlEncode(salt) + "$" + base64UrlEncode(digest);
}

```

What's happening: N=16384, r=8, p=1 with a 64-byte derived key is a deliberately slow, memory-hard setting that blunts brute force. The 16-byte salt from SecureRandom defeats rainbow tables and makes identical passwords hash differently. Output format is the literal string `scrypt$salt$digest` in URL-safe base64. `derive()` delegates to Bouncy Castle's `SCrypt.generate` with UTF-8 password bytes.

Why it matters: Registration stores this string; nothing else in the system can reverse it. The format is also compatible with the project's older Python hashes (proven by test).

Byte 27: SecurityService — verifying a password

Builds on: Byte 26

In plain terms: Login re-derives the digest with the stored salt and compares in constant time, rejecting anything malformed.

The code:

```

public boolean verifyPassword(String password, String storedHash) {
    if (storedHash == null) return false;
    String[] parts = storedHash.split("\\$", -1);
    if (parts.length != 3 || !"scrypt".equals(parts[0])) return false;
    try {
        byte[] salt = base64UrlDecode(parts[1]);
        byte[] expected = base64UrlDecode(parts[2]);
        byte[] actual = derive(password, salt);
        return MessageDigest.isEqual(actual, expected);
    } catch (Exception e) {
        return false;
    }
}

```

What's happening: Parameters: the candidate password and the stored `scrypt$...` string. The split on literal `$` must yield exactly three parts with the 'scrypt' prefix, else false — so legacy or corrupt hashes fail closed. `MessageDigest.isEqual` compares in constant time to avoid timing side-channels. Any exception (bad base64, nulls) also returns false rather than throwing, so login simply yields 401.

Why it matters: A wrong password and a corrupt hash are indistinguishable to callers: both just fail. That fail-closed posture is the point.

Byte 28: SecurityService — session token lifecycle

Builds on: Bytes 9, 21

In plain terms: Login sessions ride on 256-bit random tokens; the database keeps only their SHA-256 fingerprint.

The code:

```
public String createSessionToken() {
    byte[] bytes = new byte[32];
    random.nextBytes(bytes);
    return base64UrlEncode(bytes);
}

public String hashSessionToken(String token) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(token.getBytes(StandardCharsets.UTF_8));
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) sb.append(String.format("%02x", b));
        return sb.toString();
    } catch (Exception e) {
        throw new IllegalStateException(e);
    }
}
```

What's happening: createSessionToken needs no parameters and returns a 32-byte (256-bit) unguessable token as URL-safe text — safe to embed in a cookie. hashSessionToken takes that token and returns lowercase hex SHA-256; AuthService stores this hash and re-hashes the presented cookie for comparison. SHA-256 is always available, so the catch branch is theoretically unreachable (wrapped as unchecked to keep signatures clean).

Why it matters: Stealing the sessions collection buys an attacker nothing usable: raw tokens exist only in browsers and transit.

Byte 29: SecurityService — key derivation and base64 helpers

Builds on: Bytes 26–28

In plain terms: One private worker runs scrypt; two package-visible helpers convert binary to text and back, tolerating both base64 alphabets.

The code:

```
private static byte[] derive(String password, byte[] salt) {
    return SCrypt.generate(password.getBytes(StandardCharsets.UTF_8), salt, SCRYPT_N,
        SCRYPT_R, SCRYPT_P, DK_LEN);
}

static String base64UrlEncode(byte[] value) {
```

```

        return java.util.Base64.getUrlEncoder().withoutPadding().encodeToString(value);
    }
    static byte[] base64UrlDecode(String value) {
        String padded = value.replace('-', '+').replace('_', '/');
        int rem = padded.length() % 4;
        if (rem == 2) padded += "==";
        else if (rem == 3) padded += "=";
        else if (rem == 1) throw new IllegalArgumentException("Invalid base64 length");
        return java.util.Base64.getDecoder().decode(padded);
    }
}

```

What's happening: `derive` is the single choke point for script parameters — tune cost here once. `base64UrlEncode` emits unpadded URL-safe text (cookie/header friendly). `base64UrlDecode` accepts both URL-safe and standard alphabets by translating `-/_` back, then re-adds padding; a length remainder of 1 is mathematically impossible for valid base64, hence the explicit rejection. Package visibility (no modifier) lets tests exercise them without exposing them as API.

Why it matters: Hash compatibility across Python and Java eras depends on these exact byte conversions — change them and old passwords stop verifying.

backend/src/main/java/com/phonebook/service/AuthService.java

Byte 30: AuthService — cookie name, fields, constructor

Builds on: Bytes 9, 18, 21

In plain terms: The session authority: a named cookie plus three collaborators that cover sessions, users, and crypto.

The code:

```

package com.phonebook.service;
import com.phonebook.model.AuthSession;
import com.phonebook.model.User;
import com.phonebook.repo.AuthSessionRepository;
import com.phonebook.repo.UserRepository;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServletRequest;
import java.time.Duration;
import java.time.Instant;
import java.util.Optional;
import org.springframework.http.ResponseCookie;
import org.springframework.stereotype.Service;

@Service
public class AuthService {
    public static final String COOKIE_NAME = "phonebook_session";
}

```

```

    private final AuthSessionRepository sessions;
    private final UserRepository users;
    private final SecurityService security;
    public AuthService(AuthSessionRepository sessions, UserRepository users,
SecurityService security) {
        this.sessions = sessions;
        this.users = users;
        this.security = security;
    }

```

What's happening: COOKIE_NAME is public so controllers, tests, and helpers reference one spelling. Constructor injection wires the session store, user store, and crypto service — the class holds no request state, so one shared instance serves all threads. Its dependency arrows are AuthService → AuthSessionRepository, UserRepository, SecurityService.

Why it matters: Every auth flow below is a different combination of these three collaborators plus this cookie name.

Byte 31: AuthService.currentUser — the authentication checkpoint

Builds on: Byte 31

In plain terms: Turns the browser's cookie into the current User through four gates; anything suspicious yields empty.

The code:

```

public Optional<User> currentUser(HttpServletRequest request) {
    String token = sessionToken(request);
    if (token == null) return Optional.empty();
    String hash = security.hashSessionToken(token);
    Optional<AuthSession> session = sessions.findByTokenHash(hash);
    if (session.isEmpty()) return Optional.empty();
    if (session.get().getExpiresAt() == null ||
session.get().getExpiresAt().isBefore(Instant.now())) {
        return Optional.empty();
    }
    return users.findById(session.get().getUserId());
}

```

What's happening: Parameter: the raw servlet request (only its cookies are read). Gates in order: cookie present → session row exists for its hash → expiresAt set and in the future → owning user still exists. Expired sessions return empty but are NOT deleted here (no cleanup side effect). Returns Optional<User>; every controller's requireUser converts empty to HTTP 401.

Why it matters: This is the single checkpoint guarding all 20+ protected endpoints. Session lifetime (7 days) is enforced by the expiresAt comparison.

Byte 32: AuthService.sessionToken — reading the cookie

Builds on: Byte 32

In plain terms: Extracts the raw session token from the request cookies, tolerating missing or cookie-less requests.

The code:

```
public static String sessionToken(HttpServletRequest request) {
    if (request.getCookies() == null) return null;
    for (Cookie c : request.getCookies()) {
        if (COOKIE_NAME.equals(c.getName())) return c.getValue();
    }
    return null;
}
```

What's happening: Static because it needs no collaborators — pure request inspection. `getCookies()` returns null (not an empty array) when the client sent none, hence the first guard. It returns the first cookie whose name matches exactly, or null. Callers: `currentUser` and `revokeSession`.

Why it matters: All cookie parsing lives here, so a future cookie-name change touches one constant, not every endpoint.

Byte 33: AuthService.issueSession — logging in

Builds on: Bytes 21, 32

In plain terms: Mints a fresh session row and the matching cookie value with a 7-day life.

The code:

```
public ResponseCookie issueSession(Long userId) {
    String token = security.createSessionToken();
    AuthSession session = new AuthSession();
    session.setTokenHash(security.hashSessionToken(token));
    session.setUserId(userId);
    session.setExpiresAt(Instant.now().plus(Duration.ofDays(7)));
    session.setCreatedAt(Instant.now());
    sessions.save(session);
    return sessionCookie(token, Duration.ofDays(7));
}
```

What's happening: Parameter: the authenticated user's numeric id. A new random token is generated; only its hash is persisted while the raw token goes back in the cookie — the server never stores anything replayable. Expiry and cookie max-age agree (7 days). Returns a

ResponseCookie the controller attaches via the Set-Cookie header. Note the AuthSession.id is left null so Mongo assigns the default key.

Why it matters: Register and login both end here. Each call creates an independent session, so multiple browsers stay logged in separately.

Byte 34: AuthService — logout and cookie shaping

Builds on: Bytes 33, 34

In plain terms: Logout deletes the server session and erases the browser cookie; two factories define the cookie's security flags.

The code:

```
public void revokeSession(HttpServletRequest request) {
    String token = sessionToken(request);
    if (token != null) {
        sessions.deleteByTokenHash(security.hashSessionToken(token));
    }
}

public static ResponseCookie sessionCookie(String token, Duration maxAge) {
    return ResponseCookie.from(COOKIE_NAME, token)
        .httpOnly(true)    // JavaScript cannot read it (XSS protection)
        .sameSite("Lax")   // top-level navigation CSRF protection
        .secure(false)     // plain HTTP friendly; must become true behind
production HTTPS
        .path("/")
        .maxAge(maxAge)
        .build();
}

public static ResponseCookie clearCookie() {
    return ResponseCookie.from(COOKIE_NAME, "")
        .httpOnly(true).sameSite("Lax").secure(false).path("/").maxAge(0).build();
}
```

What's happening: revokeSession is null-safe: cookie-less logouts simply do nothing, and unknown tokens delete nothing. HttpOnly blocks document.cookie theft; SameSite=Lax blocks most cross-site request forgery while keeping normal logins working. clearCookie reuses identical flags with Max-Age=0 so browsers actually overwrite the stored cookie.

Why it matters: Logout calls revokeSession then sends clearCookie. The secure=false line is the known production hardening item.

backend/src/main/java/com/phonebook/service/Validators.java

Byte 35: Validators — patterns and the unused FieldError

Builds on: Bytes 6, 8

In plain terms: One file holds every business rule as compiled regexes, plus a small error carrier nobody ended up using.

The code:

```
package com.phonebook.service;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.regex.Pattern;

public final class Validators {
    private Validators() {} // static utility – never instantiated
    private static final Pattern TAG_PATTERN = Pattern.compile("[A-Za-zÀ-ÿ0-9][A-Za-zÀ-ÿ0-9\\s'-]{0,49}$");
    private static final Pattern NAME_PATTERN = Pattern.compile("[A-Za-zÀ-ÿ][A-Za-zÀ-ÿ\\s'-]{1,99}$");
    private static final Pattern PHONE_ALLOWED = Pattern.compile("^\\+?[0-9()\\s-]+$");
    private static final Pattern ADDRESS_ALLOWED = Pattern.compile("[A-Za-zÀ-ÿ0-9\\s,.'#/-]+$");
    private static final Pattern HAS_LETTER = Pattern.compile("[A-Za-zÀ-ÿ]");
    private static final Pattern SCIENTIFIC = Pattern.compile("[+-]?(?:\\d+(?:\\.\\d*)?|\\.\\d+)[eE][+-]?\\d+$");
    private static final Pattern EXCEL_PHONE = Pattern.compile("^=\\\"([^\"]*)\\\"$");
    public static class FieldError {
        public final Set<String> fields = new HashSet<>();
        public final List<String> messages = new ArrayList<>();
    }
}
```

What's happening: Patterns are compiled once as constants (regex compile is expensive). À-ÿ ranges admit accented Latin letters for Indian/European names. TAG_PATTERN allows a leading digit while NAME_PATTERN demands a leading letter; PHONE_ALLOWED permits one leading + plus digits/spaces/hyphens/parens; SCIENTIFIC/EXCEL_PHONE detect spreadsheet-mangled numbers. UNUSED CODE: FieldError is public but never referenced — collectContactErrors uses a plain String plus an output Set instead.

Why it matters: Every rule below is one of these patterns applied. FieldError can be deleted with no effect.

Byte 36: Validators.validateTagName

Builds on: Byte 36

In plain terms: Accepts 1–50 character tag names of letters, numbers, spaces, apostrophes, or hyphens — nothing else.

The code:

```

public static String validateTagName(String raw) {
    String cleaned = raw != null ? raw.trim() : "";
    if (cleaned.length() < 1 || cleaned.length() > 50 ||
!TAG_PATTERN.matcher(cleaned).matches()) {
        return "Tag name must be 1 to 50 characters and contain only letters, numbers,
spaces, apostrophes or hyphens.";
    }
    return null;
}

```

What's happening: Parameter: the raw submitted name (may be null). Trimming happens before measuring, so ' Family ' validates as 'Family' — but note this method does NOT return the trimmed value; callers must re-trim (TagController does). Returns an error string or null when valid; null-for-valid is the file's convention. Commas, #, @, and emoji all fail the pattern.

Why it matters: Tag create and rename both call this first, mapping non-null to HTTP 422. The same message reaches the UI verbatim.

Byte 37: Validators.collectContactErrors — the rules orchestrator

Builds on: Bytes 7, 36

In plain terms: Validates a whole contact at once, collecting every violation and noting which fields failed, with a partial mode for updates.

The code:

```

public static String collectContactErrors(String name, String phone, String email,
String address, Set<String> fields, boolean partial) {
    List<String> messages = new ArrayList<>();
    if (!partial || name != null) {          // partial mode skips absent (null)
fields
        if (name == null || name.trim().isEmpty() ||
!NAME_PATTERN.matcher(name.trim()).matches()) {
            fields.add("name");
            messages.add("Name must contain only letters, spaces, apostrophes or
hyphens.");
        }
    }
    if (!partial || phone != null) {
        if (phone == null || phone.trim().isEmpty() ||
!PHONE_ALLOWED.matcher(phone.trim()).matches()) {
            fields.add("phone_number");
            messages.add("Phone number may contain digits, a leading +, spaces,
hyphens, or parentheses.");
        } else {
            String digits = phone.replaceAll("\\D", "");
            if (digits.length() < 8 || digits.length() > 15) {
                fields.add("phone_number");
                messages.add("Phone number must contain between 8 and 15 digits.");
            }
        }
    }
}

```

```

    }
}
if (email != null && !email.trim().isEmpty()) { // optional: blank means absent
    if (!isValidEmail(email.trim())) { fields.add("email"); messages.add("Invalid
email address."); }
}
if (address != null && !address.trim().isEmpty()) {
    String cleaned = address.trim();
    if (cleaned.length() < 5 || cleaned.length() > 255) { fields.add("address");
messages.add("Address must be between 5 and 255 characters."); }
    else if (!HAS_LETTER.matcher(cleaned).find()) { fields.add("address");
messages.add("Address must contain at least one letter."); }
    else if (!ADDRESS_ALLOWED.matcher(cleaned).matches()) { fields.add("address");
messages.add("Address contains invalid characters."); }
}
return messages.isEmpty() ? null : String.join("; ", messages);
}

```

What's happening: Parameters: the four raw values, an output set receiving failed field names, and partial (true for updates: null means 'not provided, skip'). Name/phone are required in create mode; email/address validate only when non-blank. Phone digit-counting strips non-digits first, so '+1 555 123 4567' counts 11. Returns null when clean, else semicolon-joined messages. Callers: contact create, contact update, and every CSV row — plus the fields set feeds import per-field tallies.

Why it matters: Change a rule here and forms, API, and importer all move together. That single funnel is the file's whole reason to exist.

Byte 38: Validators.isValidEmail

Builds on: Byte 38

In plain terms: A pragmatic email sanity check without regex: one @, a dotted domain, no spaces or dot abuse.

The code:

```

public static boolean isValidEmail(String email) {
    // Basic email check (local part, @, domain with a dot)
    if (email == null || email.isBlank()) return false;
    if (email.contains(" ")) return false;
    int at = email.indexOf('@');
    if (at <= 0 || at != email.lastIndexOf('@') || at == email.length() - 1) return
false;
    String domain = email.substring(at + 1);
    if (!domain.contains(".")) return false;
    if (domain.startsWith(".") || domain.endsWith(".") || domain.contains("..")) return
false;
    String local = email.substring(0, at);
    if (local.isEmpty() || local.startsWith(".") || local.endsWith(".")) return false;
}

```

```
    return true;
}
```

What's happening: Parameter: a trimmed candidate; returns boolean. It enforces exactly one @ with non-empty sides, a domain containing a dot but not leading/trailing/double dots, and a local part not starting/ending with a dot. It deliberately does NOT fully implement RFC email rules — 'a@b.co' passes while exotic-but-legal addresses may fail — trading completeness for readable error behavior.

Why it matters: Used by registration, contact create/update, and CSV import. Anything it rejects surfaces as 'Invalid email address.' with 422.

Byte 39: `Validators.normalizeHeader`

Builds on: Byte 38

In plain terms: Turns any CSV header spelling into a canonical key so 'Phone Number', 'phone-number', and 'PHONE' all match.

The code:

```
public static String normalizeHeader(String value) {
    String lower = value.trim().toLowerCase();
    String replaced = lower.replaceAll("[^a-z0-9]+", "_");
    return replaced.replaceAll("^_+|_+$", "");
}
```

What's happening: Parameter: one raw header cell. Lowercasing plus collapsing every non-alphanumeric run into a single underscore (then stripping edge underscores) maps 'First Name' → first_name, 'Mobile-Number' → mobile_number, 'EMAIL' → email. The importer compares these keys against alias sets (name/full_name/first_name/...), which is how foreign CSVs work without a mapping UI.

Why it matters: Without this, only exact lowercase headers would import. With it, the alias tables stay small while acceptance stays wide.

Byte 40: `Validators.normalizePhone`

Builds on: Bytes 38, 40

In plain terms: Unwraps Excel-mangled phone text and refuses scientific notation that cannot be recovered safely.

The code:

```
public static String normalizePhone(String value) {
    String phone = value.trim();
    var m = EXCEL_PHONE.matcher(phone);
```

```

        if (m.matches()) phone = m.group(1).trim();          //="+1415..." → +1415...
        else if (phone.startsWith("'")) phone = phone.substring(1).trim(); // leading
        apostrophe
        if (SCIENTIFIC.matcher(phone).matches()) {
            throw new IllegalStateException("Phone number is in scientific notation and
cannot be recovered safely.");
        }
        return phone;
    }
}

```

What's happening: Parameter: the raw phone cell; returns cleaned text or throws `IllegalStateException` for scientific notation. Excel exports long numbers as `"..."` text formulas or `'...-prefixed` text and as `9.11235E+11` floats; the first two are losslessly unwrapped, the third is lossy (digits are gone) so it throws and the importer counts an `invalid_phone_number` instead of guessing. The exporter deliberately writes `"..."` form, making this its mirror.

Why it matters: This is the round-trip glue between export and import. The exception type is caught by the import loop and converted into a row error.

backend/src/main/java/com/phonebook/web/ApiException.java

Byte 41: ApiException — carrying status plus body

Builds on: Byte 13

In plain terms: The one exception controllers throw to fail a request with a chosen HTTP status and JSON body.

The code:

```

package com.phonebook.web;
import org.springframework.http.HttpStatus;

public class ApiException extends RuntimeException {
    private final HttpStatus status;
    private final Object body;
    public ApiException(HttpStatus status, String detail) {
        super(detail);
        this.status = status;
        this.body = new com.phonebook.dto.Dtos.ErrorResponse(detail);
    }
    public ApiException(HttpStatus status, Object body, String message) {
        super(message);
        this.status = status;
        this.body = body;
    }
}

```

What's happening: Extending RuntimeException means no throws declarations pollute controller signatures. The two-arg constructor (status + detail string) wraps the message in ErrorResponse({detail}) — the common case. The three-arg constructor carries an arbitrary body (registration's 422 detail array) with a separate log message. Both store the HttpStatus that becomes the response code.

Why it matters: 401/404/409/422 responses throughout the codebase are all throw new ApiException(...) one-liners thanks to this class.

Byte 42: ApiException — getters

Builds on: Byte 42

In plain terms: Two accessors expose the status and body to the global handler.

The code:

```
public HttpStatus getStatus() { return status; }
public Object getBody() { return body; }
```

What's happening: No parameters, no logic — final fields set once in the constructors. The handler calls both: status becomes the response code, body becomes the serialized JSON.

Why it matters: Immutable-by-construction: once thrown, an ApiException cannot be altered in flight.

backend/src/main/java/com/phonebook/web/ApiExceptionHandler.java

Byte 43: ApiExceptionHandler — global error translation

Builds on: Bytes 42, 43

In plain terms: A single class that converts every ApiException into the HTTP response the frontend expects.

The code:

```
package com.phonebook.web;
import java.util.Map;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(ApiException.class)
    public ResponseEntity<Object> handle(ApiException ex) {
        return ResponseEntity.status(ex.getStatus()).body(ex.getBody());
    }
}
```

```
}  
}
```

What's happening: `@RestControllerAdvice` registers it for all controllers; `@ExceptionHandler(ApexException.class)` routes matching throws here instead of Spring's default error page. Note the `java.util.Map` import is unused — harmless leftover. Uncaught non-`ApiExceptions` (e.g. JSON parse failures) still fall through to Spring's default timestamp/status error format, which is why malformed JSON looks different from `{detail}` errors.

Why it matters: Controllers never build error responses by hand — they throw, and this handler renders.

backend/src/main/java/com/phonebook/web/RootController.java

Byte 44: `RootController.root` — health check

Builds on: Bytes 2, 44

In plain terms: GET / answers 'is the API alive' with a one-key JSON message.

The code:

```
package com.phonebook.web;  
import java.util.Map;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;  
  
@RestController  
public class RootController {  
    @GetMapping("/")  
    public Map<String, String> root() {  
        return Map.of("message", "Phonebook API is running");  
    }  
}
```

What's happening: No parameters, no auth, no database — Spring serializes the immutable map to `{message: ...}` with 200. `@RestController` combines `@Controller` + `@ResponseBody` so the return value is JSON, not a view name.

Why it matters: Smoke tests and the first auth test hit this to prove routing and JSON work before anything stateful runs.

backend/src/main/java/com/phonebook/web/AuthController.java

Byte 45: AuthController — fields, constructor, string helper

Builds on: Bytes 15, 19, 20, 23

In plain terms: Holds the collaborators for all four auth endpoints plus a null-safe object-to-string helper.

The code:

```
package com.phonebook.web;
// imports: Dtos, User, ContactRepository, UserRepository, AuthService,
SecurityService,
// SequenceService, Validators, HttpServletRequest, Instant, collections,
MongoTemplate/Criteria/Query/Update ...
@RestController
public class AuthController {
    private final UserRepository users;
    private final ContactRepository contacts;
    private final MongoTemplate mongoTemplate;
    private final SecurityService security;
    private final AuthService auth;
    private final SequenceService sequences;
    public AuthController(UserRepository users, ContactRepository contacts,
MongoTemplate mongoTemplate,
                        SecurityService security, AuthService auth, SequenceService
sequences) {
        this.users = users; this.contacts = contacts; this.mongoTemplate =
mongoTemplate;
        this.security = security; this.auth = auth; this.sequences = sequences;
    }
    private static String str(Object o) {
        return o == null ? null : String.valueOf(o);
    }
}
```

What's happening: Constructor injection wires six collaborators: user/contact stores, raw Mongo access (for orphan adoption), hashing, sessions, and IDs. ContactRepository itself is never used in this file (only mongoTemplate + users are) — a minor leftover. str() converts raw Map values to String while preserving null, so missing JSON keys stay null instead of becoming the string 'null'.

Why it matters: Dependency arrows: AuthController → UserRepository, MongoTemplate, SecurityService, AuthService, SequenceService.

Byte 46: AuthController.register — validation

Builds on: Bytes 15, 39, 46

In plain terms: Registration first collects all field violations into a 422 detail array before touching the database.

The code:

```
@PostMapping("/{auth/register}")
public ResponseEntity<?> register(@RequestBody Map<String, Object> raw,
    HttpServletRequest http) {
    String username = str(raw.get("username"));
    String email = str(raw.get("email"));
    String password = str(raw.get("password"));
    List<Map<String, Object>> errors = new ArrayList<>();
    String u = username != null ? username.trim() : "";
    if (u.length() < 3 || u.length() > 100) {
        errors.add(Map.of("loc", List.of("body", "username"),
            "msg", "Username must be between 3 and 100 characters.", "type",
"value_error"));
    }
    if (email == null || email.trim().isEmpty() ||
!Validators.isValidEmail(email.trim())) {
        errors.add(Map.of("loc", List.of("body", "email"),
            "msg", "Invalid email address.", "type", "value_error"));
    }
    if (password == null || password.length() < 8) {
        errors.add(Map.of("loc", List.of("body", "password"),
            "msg", "Password must contain at least 8 characters.", "type",
"value_error"));
    }
    if (!errors.isEmpty()) {
        return
ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(Map.of("detail", errors));
    }
}
```

What's happening: Parameters: the raw JSON map and the servlet request (unused here but kept for symmetry). All three rules run unconditionally so the client sees every problem at once; the detail items use FastAPI-style {loc, msg, type} shapes (a historical convention, not a Dtos record — ValidationError sits unused). Note password length is measured untrimmed while username/email are trimmed. Any failure returns 422 directly instead of throwing, because the body shape differs from ErrorResponse.

Why it matters: The register form's per-field errors come from this block. Pass it and the method proceeds to duplicate checks and creation.

Byte 47: AuthController.register — duplicates, creation, first-user adoption

Builds on: Byte 47

In plain terms: With validation passed, registration rejects taken names, hashes the password, stores the user, and logs them in immediately.

The code:

```
String cleanUsername = username.trim();
String cleanEmail = email.trim();
if (users.existsByUsername(cleanUsername)) {
    throw new ApiException(HttpStatus.CONFLICT, "Username already exists.");
}
if (users.existsByEmail(cleanEmail)) {
    throw new ApiException(HttpStatus.CONFLICT, "Email already exists.");
}
User user = new User();
user.setId(sequences.next("users"));
user.setUsername(cleanUsername);
user.setEmail(cleanEmail);
user.setPasswordHash(security.hashPassword(password));
user.setCreatedAt(Instant.now());
try {
    users.save(user);
} catch (Exception e) {
    // race: re-check
    if (users.existsByUsername(cleanUsername)) throw new
ApiException(HttpStatus.CONFLICT, "Username already exists.");
    if (users.existsByEmail(cleanEmail)) throw new
ApiException(HttpStatus.CONFLICT, "Email already exists.");
    throw e;
}
if (users.count() == 1) {
    // adopt orphan contacts (userId == null) to first user
    mongoTemplate.updateMulti(new Query(Criteria.where("userId").is(null)),
        new Update().set("userId", user.getId()),
com.phonebook.model.Contact.class);
}
var cookie = auth.issueSession(user.getId());
return ResponseEntity.status(HttpStatus.CREATED)
    .header("Set-Cookie", cookie.toString())
    .body(new Dtos.UserResponse(user.getId(), user.getUsername(),
user.getEmail(), user.getCreatedAt()));
```

What's happening: Friendly 409s come from the existsBy checks; the catch block re-checks after a save failure so a lost race still reports 409 instead of 500 (with unique DB indexes as the final guard). IDs come from SequenceService; the password is hashed, never stored. The users.count()==1 branch adopts pre-existing ownerless contacts to the very first account — a migration convenience for databases seeded before auth existed. Success returns 201 with the UserResponse plus a Set-Cookie session: registration logs you in with no second call.

Why it matters: Edge cases: simultaneous duplicate registrations → one 201, one 409. Registering the first user on a populated DB → orphans gain an owner.

Byte 48: AuthController.login — identifier plus password

Builds on: Bytes 15, 20, 22

In plain terms: One endpoint accepts a username OR an email plus a password, and answers 200 with a session or an indistinguishable 401.

The code:

```
@PostMapping("/{auth/login"})
public ResponseEntity<?> login(@RequestBody Map<String, Object> raw, HttpServletRequest
http) {
    String identifier = str(raw.get("identifier"));
    String password = str(raw.get("password"));
    if (identifier == null) identifier = "";
    User user = users.findByUsernameOrEmail(identifier, identifier).orElse(null);
    // findByUsernameOrEmail with same value twice works for derived query? Spring Data
    interprets as username=?1 OR email=?2
    if (user == null || password == null || !security.verifyPassword(password,
    user.getPasswordHash())) {
        throw new ApiException(HttpStatus.UNAUTHORIZED, "Invalid username/email or
        password.");
    }
    var cookie = auth.issueSession(user.getId());
    return ResponseEntity.ok()
        .header("Set-Cookie", cookie.toString())
        .body(new Dtos.UserResponse(user.getId(), user.getUsername(),
        user.getEmail(), user.getCreatedAt()));
}
```

What's happening: The same identifier fills both derived-query slots (username=? OR email=?). Unknown user, missing password, and wrong password all collapse into one 401 message so attackers cannot enumerate accounts. Success issues a fresh session (old sessions are NOT revoked — multi-device login stays valid) and returns 200 with the user plus Set-Cookie.

Why it matters: The sign-in form posts {identifier, password} here. Note the frontend sends identifier for login but username/email/password for register — the two shapes differ.

Byte 49: AuthController.me and logout

Builds on: Bytes 22, 23, 32

In plain terms: Two one-purpose endpoints: prove who this cookie belongs to, and destroy the session.

The code:

```
@GetMapping("/{auth/me}")
public ResponseEntity<?> me(HttpServletRequest http) {
    var user = auth.currentUser(http).orElse(null);
}
```

```

        if (user == null) throw new ApiException(HttpStatus.UNAUTHORIZED, "Authentication
required.");
        return ResponseEntity.ok(new Dtos.UserResponse(user.getId(), user.getUsername(),
user.getEmail(), user.getCreatedAt()));
    }

@PostMapping("/{auth/logout}")
public ResponseEntity<?> logout(HttpServletRequest http) {
    auth.revokeSession(http);
    return ResponseEntity.ok()
        .header("Set-Cookie", AuthService.clearCookie().toString())
        .body(new Dtos.MessageResponse("Logged out successfully."));
}

```

What's happening: `me()` takes only the request, resolves the user through the four-gate `currentUser` check, and returns 200 or 401 — the frontend calls it on every page load to restore sessions. `logout()` tolerates missing/unknown cookies (`revokeSession` is null-safe) yet always clears the browser cookie and returns the logout message with 200. Neither endpoint takes a body.

Why it matters: Session restore on refresh is `me()`; the Log out button is `logout()`. After logout, the same cookie yields 401 everywhere.

backend/src/main/java/com/phonebook/web/ContactController.java

Byte 50: ContactController — fields, constructor, auth gate

Builds on: Bytes 14, 16, 17, 22

In plain terms: The busiest controller's wiring plus the one-line method that locks every endpoint to its owner.

The code:

```

@RestController
public class ContactController {
    private final ContactRepository contacts;
    private final TagRepository tags;
    private final AuthService auth;
    private final SequenceService sequences;
    private final MongoTemplate mongoTemplate;
    public ContactController(ContactRepository contacts, TagRepository tags,
AuthService auth,
                            SequenceService sequences, MongoTemplate mongoTemplate) {
        this.contacts = contacts; this.tags = tags; this.auth = auth;
        this.sequences = sequences; this.mongoTemplate = mongoTemplate;
    }
    private User requireUser(HttpServletRequest http) {

```

```

        return auth.currentUser(http).orElseThrow(() -> new
        ApiException(HttpStatus.UNAUTHORIZED, "Authentication required."));
    }

```

What's happening: Constructor injection wires contact/tag stores, sessions, ID minting, and raw Mongo access for search. `requireUser` unwraps the `Optional` from `currentUser` or throws the uniform 401 every protected method starts with — its dependency arrow is `ContactController` → `AuthService` → (sessions, users, crypto). Note the unused import `org.springframework.data.domain.Sort` at the file top (sorting is done in memory, not via `Sort`).

Why it matters: All eight endpoints below open with `User user = requireUser(http)`. No user, no data — that invariant starts here.

Byte 51: `ContactController.list` — signature and page validation

Builds on: Bytes 12, 51

In plain terms: The list endpoint's contract: page/limit with defaults, optional search text, repeatable `tag_ids` — and strict range rules.

The code:

```

@GetMapping({"/contacts", "/contacts/"})
public Dtos.ContactPage list(
    HttpServletRequest http,
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(defaultValue = "10") int limit,
    @RequestParam(required = false) String search,
    @RequestParam(name = "tag_ids", required = false) List<Long> tagIds) {
    User user = requireUser(http);
    if (page < 1 || limit < 1 || limit > 100) {
        throw new ApiException(HttpStatus.UNPROCESSABLE_ENTITY, "Page must be positive
and limit must be between 1 and 100.");
    }
}

```

What's happening: Both `/contacts` and `/contacts/` route here so trailing slashes never 404. page/limit default to 1/10; search is optional free text; `tag_ids` may repeat (`?tag_ids=3&tag_ids=5`) and Spring binds it to a list. Out-of-range paging is a 422 (not a silent clamp), capping pages at 100 rows to protect the database. Returns `ContactPage` on success.

Why it matters: The frontend's search box, tag chips, and page controls all feed these four parameters.

Byte 52: `ContactController.list` — building the filter

Builds on: Byte 52

In plain terms: Three composable Mongo criteria — owner, text search, must-have-all-tags — ANDed into one query.

The code:

```
Criteria criteria = Criteria.where("userId").is(user.getId());
if (search != null && !search.trim().isEmpty()) {
    String s = Pattern.quote(search.trim());
    Pattern rx = Pattern.compile(".*" + s + ".*", Pattern.CASE_INSENSITIVE);
    criteria = new Criteria().andOperator(criteria, new Criteria().orOperator(
        Criteria.where("name").regex(rx),
        Criteria.where("phoneNumber").regex(rx),
        Criteria.where("email").regex(rx)));
}
List<Long> distinctTags = tagIds != null ? tagIds.stream().distinct().toList() :
List.of();
if (!distinctTags.isEmpty()) {
    criteria = new Criteria().andOperator(criteria,
Criteria.where("tagIds").all(distinctTags));
}
```

What's happening: Owner scoping is always present — isolation is structural, not optional. Search becomes a case-insensitive substring regex ORed across name/phone/email; Pattern.quote neutralizes regex metacharacters so searching '(555)' cannot break or hijack the query. Tag filtering uses \$all semantics: the contact must embed every selected tag (AND, not OR). Duplicate tag IDs are deduped first so counts stay correct.

Why it matters: This block is the entire search-and-filter specification. Blank search or no tags simply skip their clauses.

Byte 53: `ContactController.list` — count, sort, slice, envelope

Builds on: Byte 53

In plain terms: Counts the total, sorts alphabetically in memory, slices one page, and wraps it in the pagination envelope.

The code:

```
Query countQuery = new Query(criteria);
long total = mongoTemplate.count(countQuery, Contact.class);
// Fetch all matching, sort by name asc in memory (case-sensitive ordinal)
Query query = new Query(criteria);
List<Contact> all = mongoTemplate.find(query, Contact.class);
all.sort(Comparator.comparing(c -> c.getName() != null ? c.getName() : "",
String::compareTo));
int totalPages = Math.max((int) ((total + limit - 1) / limit), 1);
int from = Math.min((page - 1) * limit, all.size());
int to = Math.min(from + limit, all.size());
List<Contact> slice = all.subList(from, to);
```

```
List<Dtos.ContactResponse> items = toResponses(slice);
return new Dtos.ContactPage(items, page, limit, total, totalPages);
```

What's happening: total counts matches for the envelope while the sort-then-slice produces the page; totalPages uses ceiling division with a minimum of 1 so empty lists still report one page. Sorting is in-memory case-sensitive ordinal (uppercase before lowercase) with null names treated as empty. from/to are clamped with min() so over-deep pages return an empty slice instead of throwing. toResponses resolves tag IDs to sorted tag objects for the slice only.

Why it matters: Gotcha: the whole match set is loaded before slicing — fine at phonebook scale, but the known scaling bottleneck if a user ever held millions of contacts.

Byte 54: ContactController.create — input extraction and validation

Builds on: Bytes 24, 51

In plain terms: Create reads both snake_case and camelCase spellings, validates business rules, and confirms the tags belong to the caller.

The code:

```
@PostMapping("/{contacts}", "/contacts/")
public ResponseEntity<?> create(@RequestBody Map<String, Object> raw,
    HttpServletRequest http) {
    User user = requireUser(http);
    String name = asString(raw.get("name"));
    String phone = asString(raw.get("phone_number"));
    if (phone == null) phone = asString(raw.get("phoneNumber"));
    String email = asString(raw.get("email"));
    String address = asString(raw.get("address"));
    List<Long> tagIds = asLongList(raw.get("tag_ids"));
    if (tagIds == null) tagIds = asLongList(raw.get("tagIds"));
    Set<String> fields = new HashSet<>();
    String err = Validators.collectContactErrors(name, phone, email, address, fields,
false);
    if (err != null) throw new ApiException(HttpStatus.UNPROCESSABLE_ENTITY, err);
    String tagErr = ensureOwnedTags(tagIds, user.getId());
    if (tagErr != null) throw new ApiException(HttpStatus.BAD_REQUEST, tagErr);
```

What's happening: Dual-spelling reads exist because Jackson's case-insensitive mapping plus older clients send either form. Validation runs in create mode (partial=false: name and phone required). ensureOwnedTags (Byte 64) rejects foreign tag IDs with 400 before anything is stored. asString preserves null for missing keys; asLongList parses numbers or numeric strings and yields -1L sentinels for garbage (which then fail the ownership check).

Why it matters: Order matters: 401 → 422 (bad values) → 400 (foreign tags) → 409 (duplicates, next byte). The frontend can rely on that precedence.

Byte 55: ContactController.create — duplicates, build, save

Builds on: Byte 55

In plain terms: With values valid, creation rejects duplicate phone/email, mints an ID, and persists — converting index races into clean 409s.

The code:

```
String cleanPhone = phone.trim();
String cleanEmail = (email == null || email.trim().isEmpty()) ? null :
email.trim();
if (contacts.existsByPhoneNumber(cleanPhone)) {
    throw new ApiException(HttpStatus.CONFLICT, "This phone number already
exists.");
}
if (cleanEmail != null && contacts.existsByEmail(cleanEmail)) {
    throw new ApiException(HttpStatus.CONFLICT, "This email address already
exists.");
}
Contact contact = new Contact();
contact.setId(sequences.next("contacts"));
contact.setUserId(user.getId());
contact.setName(name.trim());
contact.setPhoneNumber(cleanPhone);
contact.setEmail(cleanEmail);
contact.setAddress(address == null || address.trim().isEmpty() ? null :
address.trim());
contact.setCreatedAt(Instant.now());
contact.setTagIds(tagIds != null ? distinct(tagIds) : new ArrayList<>());
try {
    contacts.save(contact);
} catch (Exception e) {
    throw new ApiException(HttpStatus.CONFLICT, "A contact with this phone number
or email already exists.");
}
return ResponseEntity.status(HttpStatus.CREATED).body(toResponse(contact));
```

What's happening: Blank email/address normalize to null so the sparse email index ignores them (many contacts may omit email). Uniqueness checks are global, not per-user. The try/catch around save converts unique-index violations from a lost race into the same 409 instead of a 500. Success returns 201 with the full ContactResponse including resolved tags.

Why it matters: The Add-contact form's success path ends here. Failure precedence continues: duplicates report after validation and tag checks.

Byte 56: ContactController.exportCsv — download endpoint

Builds on: Bytes 17, 51

In plain terms: Exports the caller's full contact list as a downloadable CSV with spreadsheet-safe phone formatting.

The code:

```
@GetMapping("/contacts/export")
public ResponseEntity<?> exportCsv(HttpServletRequest http) {
    User user = requireUser(http);
    List<Contact> all = contacts.findByIdOrderByNameAsc(user.getId());
    StringBuilder sb = new StringBuilder();
    sb.append("Name,Phone number,Email,Address\n");
    for (Contact c : all) {
        sb.append(csvField(c.getName())).append(",");
        sb.append(csvField("=\"" + c.getPhoneNumber() + "\"")).append(",");
        sb.append(csvField(c.getEmail() != null ? c.getEmail() : "")).append(",");
        sb.append(csvField(c.getAddress() != null ? c.getAddress() : "")).append("\n");
    }
    byte[] bytes = sb.toString().getBytes(StandardCharsets.UTF_8);
    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment;
filename=phonebook.csv")
        .contentType(MediaType.parseMediaType("text/csv; charset=utf-8"))
        .body(bytes);
}
```

What's happening: Contacts come pre-sorted A→Z from the repository. Phones are wrapped as `="+1415..."` so Excel keeps the plus and full digits instead of converting to a number (the importer unwraps exactly this). `csvField` quotes values containing commas/quotes/newlines, doubling inner quotes. Content-Disposition: attachment forces download; only the caller's contacts are included. Route-order gotcha: this mapping is declared BEFORE `/contacts/{id}` so 'export' is never mistaken for an ID.

Why it matters: The frontend Export CSV button is a plain authenticated GET here. Tags are intentionally not exported.

Byte 57: `ContactController.importCsv` — file gate and header aliases

Builds on: Bytes 5, 24, 25, 51

In plain terms: Import first rejects non-CSV, empty, and non-UTF-8 uploads, then requires recognizable name and phone columns across many aliases.

The code:

```
@PostMapping("/contacts/import")
public ResponseEntity<?> importCsv(@RequestParam(value = "file", required = false)
MultipartFile file, HttpServletRequest http) {
    User user = requireUser(http);
    if (file == null || file.getOriginalFilename() == null ||
file.getOriginalFilename().isBlank())
```

```

        || !file.getOriginalFilename().toLowerCase().endsWith(".csv")) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "Please upload a CSV file.");
        }
        if (file.isEmpty()) throw new ApiException(HttpStatus.BAD_REQUEST, "The CSV file is empty.");
        String text;
        try {
            byte[] bytes = file.getBytes();
            java.nio.charset.CharsetDecoder decoder = StandardCharsets.UTF_8.newDecoder()
                .onMalformedInput(java.nio.charset.CodingErrorAction.REPORT)
                .onUnmappableCharacter(java.nio.charset.CodingErrorAction.REPORT);
            text = decoder.decode(java.nio.ByteBuffer.wrap(bytes)).toString();
        } catch (Exception e) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "The CSV file must use UTF-8 encoding.");
        }
        if (text.trim().isEmpty()) throw new ApiException(HttpStatus.BAD_REQUEST, "The CSV file is empty.");
        List<String> headers; List<CSVRecord> records;
        try {
            CSVParser parser = CSVParser.parse(text,
                CSVFormat.DEFAULT.withFirstRecordAsHeader().withIgnoreEmptyLines(false).withTrim(false)
            );
            headers = parser.getHeaderNames(); records = parser.getRecords();
        } catch (Exception e) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "The CSV file is empty.");
        }
        if (headers == null || headers.isEmpty()) throw new
            ApiException(HttpStatus.BAD_REQUEST, "The CSV file is empty.");
        Set<String> normalized =
            headers.stream().map(Validators::normalizeHeader).collect(Collectors.toSet());
        Set<String> nameAliases = Set.of("name", "full_name");
        Set<String> firstAliases = Set.of("first_name", "firstname");
        Set<String> lastAliases = Set.of("last_name", "lastname");
        Set<String> phoneAliases = Set.of("phone", "phone_number", "mobile",
            "mobile_number");
        boolean hasName = normalized.stream().anyMatch(nameAliases::contains)
            || normalized.stream().anyMatch(firstAliases::contains)
            || normalized.stream().anyMatch(lastAliases::contains);
        boolean hasPhone = normalized.stream().anyMatch(phoneAliases::contains);
        if (!hasName || !hasPhone) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "CSV must include a name or first/last name column and a phone column.");
        }
    }

```

What's happening: The extension gate uses the original filename (case-insensitive); missing file part also lands here. Strict UTF-8 decoding with REPORT rejects mojibake files instead of guessing. commons-csv parses with the first record as header, preserving empty lines as records and raw cell spacing. Header matching runs on normalized keys against alias sets, so 'First

Name/'/Mobile' style exports from other apps are accepted. Every failure is a 400 with a UI-ready sentence — import never returns 422.

Why it matters: This gate decides 'is this even a processable CSV' before a single contact is examined. The alias sets are the documented compatibility list.

Byte 58: `ContactController.importCsv` — row loop, dedup, summary

Builds on: Bytes 24, 25, 58

In plain terms: Each row is normalized, validated, de-duplicated, and tallied into the import report; survivors are bulk-saved once.

The code:

```
Set<String> seenPhones =
contacts.findAll().stream().map(Contact::getPhoneNumber).collect(Collectors.toSet());
Set<String> seenEmails = contacts.findAll().stream()
    .filter(c -> c.getEmail() !=
null).map(Contact::getEmail).collect(Collectors.toSet());
int totalRows = 0, imported = 0, skipped = 0, invalidRows = 0;
int invalidNames = 0, invalidPhones = 0, invalidEmails = 0, invalidAddresses = 0;
List<Dtos.ImportRowError> rowErrors = new ArrayList<>();
List<Contact> toSave = new ArrayList<>();
int rowNumber = 1;
for (CSVRecord record : records) {
    rowNumber++; totalRows++;
    String name = csvValue(record, headers, nameAliases);
    if (name == null) { // compose from first + last name columns
        String first = csvValue(record, headers, firstAliases);
        String last = csvValue(record, headers, lastAliases);
        List<String> parts = new ArrayList<>();
        if (first != null && !first.isBlank()) parts.add(first.trim());
        if (last != null && !last.isBlank()) parts.add(last.trim());
        name = parts.isEmpty() ? null : String.join(" ", parts);
    }
    String phone = csvValue(record, headers, phoneAliases);
    if (phone == null) phone = "";
    String email = csvValue(record, headers, Set.of("email", "email_address"));
    String address = csvValue(record, headers, Set.of("address",
"street_address"));
    if (address == null) { // compose from city + country columns
        String city = csvValue(record, headers, Set.of("city", "town"));
        String country = csvValue(record, headers, Set.of("country",
"country_name"));
        List<String> parts = new ArrayList<>();
        if (city != null && !city.isBlank()) parts.add(city.trim());
        if (country != null && !country.isBlank()) parts.add(country.trim());
        address = parts.isEmpty() ? null : String.join(" ", parts);
    }
    Set<String> fields = new HashSet<>();
    String error;
```

```

        try {
            phone = Validators.normalizePhone(phone == null ? "" : phone);
            error = Validators.collectContactErrors(name, phone, email, address,
fields, false);
        } catch (IllegalStateException ex) {    // scientific notation
            fields.add("phone_number");
            error = ex.getMessage();
        }
        if (error != null) {
            invalidRows++;
            if (fields.contains("name")) invalidNames++;
            if (fields.contains("phone_number")) invalidPhones++;
            if (fields.contains("email")) invalidEmails++;
            if (fields.contains("address")) invalidAddresses++;
            rowErrors.add(new Dtos.ImportRowError(rowNumber, error));
            continue;
        }
        String normEmail = (email == null || email.trim().isEmpty()) ? null :
email.trim();
        if (seenPhones.contains(phone) || (normEmail != null &&
seenEmails.contains(normEmail))) {
            skipped++;
            continue;
        }
        Contact c = new Contact();
        c.setId(sequences.next("contacts"));
        c.setUserId(user.getId());
        c.setName(name.trim());
        c.setPhoneNumber(phone);
        c.setEmail(normEmail);
        c.setAddress(address == null || address.trim().isEmpty() ? null :
address.trim());
        c.setCreatedAt(Instant.now());
        c.setTagIds(new ArrayList<>());    // imports never assign tags
        toSave.add(c);
        seenPhones.add(phone);
        if (normEmail != null) seenEmails.add(normEmail);
        imported++;
    }
    if (!toSave.isEmpty()) contacts.saveAll(toSave);
    var summary = new Dtos.ImportSummary(totalRows, imported, skipped, invalidRows,
        invalidNames, invalidPhones, invalidEmails, invalidAddresses, rowErrors);
    return ResponseEntity.ok(summary);

```

What's happening: csvValue returns trimmed non-empty cell text or null by scanning the header's normalized aliases. rowNumber starts at 1 and pre-increments, so the header is row 1 and the first data row is 2 — error rows point at the user's spreadsheet lines. seenPhones/seenEmails preload the whole collection AND grow with accepted rows, catching intra-file duplicates too. Scientific-notation phones become phone failures via the catch. New rows always belong to the uploader with empty tags. One saveAll persists everything and the

ImportSummary (always HTTP 200, even with zero imports) reports it. Note: preload uses findAll across all users, so a phone owned by someone else counts as a duplicate here.

Why it matters: The import dialog's report box is this summary object. understood deeply, this loop explains every number in it.

Byte 59: `ContactController.getOne` — reading a single contact

Builds on: Bytes 51, 64

In plain terms: Returns one contact by ID, or 404 when it does not exist or belongs to someone else.

The code:

```
@GetMapping("/contacts/{id}")
public ResponseEntity<> getOne(@PathVariable long id, HttpServletRequest http) {
    User user = requireUser(http);
    Contact contact = contacts.findByIdAndUserId(id, user.getId()).orElse(null);
    if (contact == null) return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new
Dtos.ErrorResponse("Contact not found."));
    return ResponseEntity.ok(toResponse(contact));
}
```

What's happening: Parameters: the numeric path ID (Spring converts and rejects non-numbers with its default 400) plus the request for auth. The {id} template must not shadow /contacts/export — declaration order plus exact-match precedence keeps export safe. toResponse resolves tag IDs into sorted tag objects. Direct 404 body here (not a throw) is stylistic; the outcome matches thrown 404s elsewhere.

Why it matters: The contact detail panel loads through this. Foreign IDs are indistinguishable from missing ones by design.

Byte 60: `ContactController.update` — extracting and validating a partial edit

Builds on: Bytes 24, 51

In plain terms: Update accepts any subset of fields (plus tags), validates only what was sent, and treats explicit JSON null as 'leave unchanged'.

The code:

```
@PutMapping("/contacts/{id}")
public ResponseEntity<> update(@PathVariable long id, @RequestBody Map<String, Object>
raw, HttpServletRequest http) {
    User user = requireUser(http);
    Contact contact = contacts.findByIdAndUserId(id, user.getId()).orElse(null);
    if (contact == null) return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new
Dtos.ErrorResponse("Contact not found."));
    boolean hasName = raw.containsKey("name");
```

```

        boolean hasPhone = raw.containsKey("phone_number") ||
raw.containsKey("phoneNumber");
        boolean hasEmail = raw.containsKey("email");
        boolean hasAddress = raw.containsKey("address");
        boolean hasTags = raw.containsKey("tag_ids") || raw.containsKey("tagIds");
        String name = hasName ? asString(raw.get("name")) : null;
        String phone = null;
        if (raw.containsKey("phone_number")) phone = asString(raw.get("phone_number"));
        else if (raw.containsKey("phoneNumber")) phone = asString(raw.get("phoneNumber"));
        else if (hasPhone) phone = null;
        String email = hasEmail ? asString(raw.get("email")) : null;
        String address = hasAddress ? asString(raw.get("address")) : null;
        List<Long> tagIds = null;
        if (raw.containsKey("tag_ids")) tagIds = asLongList(raw.get("tag_ids"));
        else if (raw.containsKey("tagIds")) tagIds = asLongList(raw.get("tagIds"));
        // validation: if none of the 4 contact fields present at all -> allow (tag-only
update)
        if (hasName || hasPhone || hasEmail || hasAddress) {
            Set<String> fields = new HashSet<>();
            String vName = hasName ? name : null;
            String vPhone = hasPhone ? phone : null;
            String vEmail = hasEmail ? email : null;
            String vAddress = hasAddress ? address : null;
            // partial=true but only validate present keys: pass null for absent so
validator skips
            // For present-but-null (explicit JSON null), treat as absent (skip) (null = no
change)
            String err = Validators.collectContactErrors(vName, vPhone, vEmail, vAddress,
fields, true);
            // collectContactErrors with partial=true skips nulls, so explicit null = skip.
Good.
            if (err != null) throw new ApiException(HttpStatus.UNPROCESSABLE_ENTITY, err);
        }

```

What's happening: Presence is detected with `containsKey` (not nullness), so `{name: null}` counts as present-but-null and, via partial mode, validates to a skip — null never clears a field. Tag-only updates (no contact keys at all) skip validation entirely and proceed to the tag branch. `hasTags` is computed but only used implicitly later (`tagIds != null` gate) — slight redundancy, harmless. `hasPhone`'s third branch is dead logic (`hasPhone` true with neither key is impossible) but documents intent.

Why it matters: Consequence for the UI: there is NO way to null out email/address via the API — empty string clears (next byte), null is ignored.

Byte 61: `ContactController.update` — applying the edit

Builds on: Bytes 61, 64

In plain terms: Re-checks ownership of tags and uniqueness of changed phone/email, then applies only the provided fields.

The code:

```
if (tagIds != null) {
    String tagErr = ensureOwnedTags(tagIds, user.getId());
    if (tagErr != null) throw new ApiException(HttpStatus.BAD_REQUEST, tagErr);
}
if (hasPhone && phone != null) {
    String cleanPhone = phone.trim();
    var existing = contacts.findByPhoneNumber(cleanPhone).orElse(null);
    if (existing != null && !existing.getId().equals(contact.getId())) {
        throw new ApiException(HttpStatus.CONFLICT, "This phone number already
exists.");
    }
}
if (hasEmail && email != null) {
    String cleanEmail = email.trim().isEmpty() ? null : email.trim();
    if (cleanEmail != null) {
        var existing = contacts.findByEmail(cleanEmail).orElse(null);
        if (existing != null && !existing.getId().equals(contact.getId())) {
            throw new ApiException(HttpStatus.CONFLICT, "This email address already
exists.");
        }
    }
}
if (hasName && name != null) contact.setName(name.trim());
if (hasPhone && phone != null) contact.setPhoneNumber(phone.trim());
if (hasEmail && email != null) contact.setEmail(email.trim().isEmpty() ? null :
email.trim());
if (hasAddress && address != null) contact.setAddress(address.trim().isEmpty() ?
null : address.trim());
if (tagIds != null) contact.setTagIds(distinct(tagIds));
try {
    contacts.save(contact);
} catch (Exception e) {
    throw new ApiException(HttpStatus.CONFLICT, "A contact with this phone number
or email already exists.");
}
return ResponseEntity.ok(toResponse(contact));
```

What's happening: Uniqueness re-checks exclude the contact itself (same id → allowed), so saving unchanged values never 409s. The apply block writes only present-and-non-null fields: empty string email/address clears to null, while tagIds uses null-vs-empty to distinguish 'leave tags' (absent key) from 'clear tags' (empty array). The save try/catch converts index races to 409. Returns 200 with the updated ContactResponse.

Why it matters: Edit-modal semantics in one place: partial edits, self-excluding uniqueness, null-means-ignore, empty-array-means-clear-tags.

Byte 62: [ContactController.delete](#)

Builds on: Byte 51

In plain terms: Deletes one owned contact, or 404s when it is missing or foreign.

The code:

```
@DeleteMapping("/contacts/{id}")
public ResponseEntity<?> delete(@PathVariable long id, HttpServletRequest http) {
    User user = requireUser(http);
    Contact contact = contacts.findByIdAndUserId(id, user.getId()).orElse(null);
    if (contact == null) return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new
Dtos.ErrorResponse("Contact not found."));
    contacts.delete(contact);
    return ResponseEntity.ok(new Dtos.MessageResponse("Contact deleted
successfully."));
}
```

What's happening: Same gate pattern as `getOne`: scoped lookup, 404 body, then the repository delete of the loaded entity. No cascade is needed — contacts own nothing else (tags live independently and merely lose one reference when a contact vanishes). Returns 200 with a message the UI toasts.

Why it matters: The delete-confirm button maps here. Deleting a contact never affects tags or other users.

Byte 63: `ContactController` helpers — tag ownership and response shaping

Builds on: Bytes 17, 51

In plain terms: Three private workers: prove every tag ID belongs to the caller, and translate stored tag IDs into sorted tag objects for one contact or a page.

The code:

```
private String ensureOwnedTags(List<Long> tagIds, Long userId) {
    if (tagIds == null) return null; // absent key → nothing to check
    List<Long> unique = distinct(tagIds);
    if (unique.stream().anyMatch(t -> t <= 0)) return "One or more tags were not
found.";
    if (unique.isEmpty()) return null; // empty array → clear, always allowed
    long owned = tags.countByUserIdAndIdIn(userId, unique);
    return owned == unique.size() ? null : "One or more tags were not found.";
}

private Dtos.ContactResponse toResponse(Contact c) {
    List<Dtos.TagResponse> tagResponses = new ArrayList<>();
    if (c.getTagIds() != null && !c.getTagIds().isEmpty()) {
        List<Tag> found = tags.findByIdIn(c.getTagIds());
        Map<Long, Tag> byId = new HashMap<>();
        for (Tag t : found) byId.put(t.getId(), t);
        for (Long tid : c.getTagIds()) {
            Tag t = byId.get(tid);
        }
    }
}
```

```

        if (t != null) tagResponses.add(new Dtos.TagResponse(t.getId(),
t.getName()));
    }
    tagResponses.sort(Comparator.comparing(Dtos.TagResponse::name));
}
return new Dtos.ContactResponse(c.getId(), c.getName(), c.getPhoneNumber(),
c.getEmail(),
    c.getAddress(), c.getCreatedAt(), tagResponses);
}
private List<Dtos.ContactResponse> toResponses(List<Contact> list) { /* batch version:
one findByIdIn for all tagIds on the page, then same per-contact assembly + sort */ }

```

What's happening: ensureOwnedTags returns null when valid (callers throw 400 otherwise); non-positive IDs fail fast without a query; the count comparison is one round-trip regardless of tag count. toResponse preserves the contact's tag order for existing IDs, drops dangling IDs (e.g. tags deleted concurrently), and sorts pills alphabetically. toResponses hoists the batch load: it unions all tag IDs on the page into a single findByIdIn instead of one query per contact (avoids N+1).

Why it matters: Every contact response the API emits passes through these. Ghost-tag tolerance and stable pill ordering both live here.

Byte 64: ContactController helpers — parsing and CSV utilities

Builds on: Byte 64

In plain terms: Five tiny static utilities for reading loose JSON and writing/reading CSV cells.

The code:

```

private static String asString(Object o) { return o == null ? null : String.valueOf(o);
}
private static List<Long> asLongList(Object o) {
    if (o == null) return null;
    if (o instanceof List<?> list) {
        List<Long> out = new ArrayList<>();
        for (Object e : list) {
            if (e instanceof Number n) out.add(n.longValue());
            else { try { out.add(Long.parseLong(String.valueOf(e))); }
                catch (NumberFormatException ex) { out.add(-1L); } }
        }
        return out;
    }
    return null;
}
private static List<Long> distinct(List<Long> in) { return
in.stream().distinct().toList(); }
private static String csvField(String value) {
    if (value == null) return "";
}

```

```

        boolean needQuote = value.contains(",") || value.contains("\"") ||
value.contains("\n");
        if (!needQuote) return value;
        return "\"" + value.replace("\"", "\\\"") + "\"";
    }
    private static String csvValue(CSVRecord record, List<String> headers, Set<String>
aliases) {
        for (int i = 0; i < headers.size(); i++) {
            if (aliases.contains(Validators.normalizeHeader(headers.get(i)))) {
                String v = i < record.size() ? record.get(i) : null;
                if (v != null) { v = v.trim(); return v.isEmpty() ? null : v; }
                return null;
            }
        }
        return null;
    }
}

```

What's happening: asString keeps nulls null (never the string 'null'). asLongList accepts JSON numbers and numeric strings; garbage becomes -1L, which later fails ownership checks as 'not found' rather than crashing. distinct preserves first-seen order. csvField implements minimal RFC-4180 quoting for export. csvValue finds the first header whose normalized form hits the alias set, guards short rows (i < record.size), and normalizes blanks to null.

Why it matters: Loose client JSON and ragged CSV rows are tamed here, so the endpoint bodies stay readable.

backend/src/main/java/com/phonebook/web/TagController.java

Byte 65: TagController — fields, constructor, auth gate

Builds on: Bytes 17, 19, 22

In plain terms: The tag controller's wiring mirrors the contact controller's, minus contact access.

The code:

```

package com.phonebook.web;
// imports: Dtos, Tag, User, TagRepository, AuthService, SequenceService, Validators,
// HttpServletRequest, List/Map, MongoTemplate/Criteria/Query, HTTP types, mapping
// annotations
@RestController
public class TagController {
    private final TagRepository tags;
    private final AuthService auth;
    private final SequenceService sequences;
    private final MongoTemplate mongoTemplate; // used by delete-cascade only
    public TagController(TagRepository tags, AuthService auth, SequenceService
sequences, MongoTemplate mongoTemplate) {

```

```

        this.tags = tags; this.auth = auth; this.sequences = sequences;
this.mongoTemplate = mongoTemplate;
    }
    private User requireUser(HttpServletRequest http) {
        return auth.currentUser(http).orElseThrow(() -> new
        ApiException(HttpStatus.UNAUTHORIZED, "Authentication required.));
    }
}

```

What's happening: Same requireUser 401 gate as contacts. MongoTemplate is used solely by the delete cascade (Byte 70) — list/create/update never touch it. Dependency arrows: TagController → TagRepository, AuthService, SequenceService, MongoTemplate.

Why it matters: Four endpoints below (list, create, update, delete) all start with requireUser.

Byte 66: TagController.list — alphabetical tag listing

Builds on: Bytes 17, 66

In plain terms: Returns the caller's tags A→Z as id/name pairs.

The code:

```

@GetMapping("/{tags}", "/tags/")
public List<Dtos.TagResponse> list(HttpServletRequest http) {
    User user = requireUser(http);
    return tags.findByIdOrderByUserNameAsc(user.getId()).stream()
        .map(t -> new Dtos.TagResponse(t.getId(), t.getName())).toList();
}

```

What's happening: No parameters beyond the request; no pagination (tag counts stay small). The repository's OrderByNameAsc does the sorting in MongoDB. An empty list serializes to [] with 200 — never 404.

Why it matters: Filter chips, the tag manager list, and the contact-form checkboxes all render from this array.

Byte 67: TagController.create — validating and storing a tag

Builds on: Bytes 19, 37, 66

In plain terms: Trims, validates, rejects per-user duplicates (case-insensitive), and stores with a fresh sequence ID.

The code:

```

@PostMapping("/{tags}", "/tags/")
public ResponseEntity<> create(@RequestBody Map<String, Object> raw,
    HttpServletRequest http) {

```

```

    User user = requireUser(http);
    String name = raw.get("name") == null ? null : String.valueOf(raw.get("name"));
    String err = Validators.validateTagName(name);
    if (err != null) throw new ApiException(HttpStatus.UNPROCESSABLE_ENTITY, err);
    String cleaned = name.trim();
    if (tags.existsByUserIdAndLowerName(user.getId(), cleaned.toLowerCase())) {
        throw new ApiException(HttpStatus.CONFLICT, "A tag with this name already
exists.");
    }
    Tag tag = new Tag();
    tag.setId(sequences.next("tags"));
    tag.setUserId(user.getId());
    tag.setName(cleaned); // also sets lowerName
    try {
        tags.save(tag);
    } catch (Exception e) {
        throw new ApiException(HttpStatus.CONFLICT, "A tag with this name already
exists.");
    }
    return ResponseEntity.status(HttpStatus.CREATED).body(new
Dtos.TagResponse(tag.getId(), tag.getName()));
}

```

What's happening: Validation (422) precedes the duplicate check (409); the try/catch converts a lost uniqueness race into the same 409. setName syncs lowerName for the index. Only the trimmed name is stored, so surrounding spaces never create lookalike tags. Returns 201 with the created tag.

Why it matters: The tag manager's create box posts {name} here; blank or 'bad,name' inputs fail with the exact validator message.

Byte 68: TagController.update — renaming with rival detection

Builds on: Bytes 37, 66

In plain terms: Renames one owned tag, allowing case-only changes but blocking collisions with sibling tags.

The code:

```

@PutMapping("/{tags/{id}}")
public ResponseEntity<?> update(@PathVariable long id, @RequestBody Map<String, Object>
raw, HttpServletRequest http) {
    User user = requireUser(http);
    Tag tag = tags.findByIdAndUserId(id, user.getId()).orElse(null);
    if (tag == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new
Dtos.ErrorResponse("Tag not found."));
    }
    String name = raw.get("name") == null ? null : String.valueOf(raw.get("name"));
    String err = Validators.validateTagName(name);

```

```

        if (err != null) throw new ApiException(HttpStatus.UNPROCESSABLE_ENTITY, err);
        String cleaned = name.trim();
        var existing = tags.findByUserIdAndLowerName(user.getId(),
cleaned.toLowerCase()).orElse(null);
        if (existing != null && !existing.getId().equals(tag.getId())) {
            throw new ApiException(HttpStatus.CONFLICT, "A tag with this name already
exists.");
        }
        tag.setName(cleaned);
        try {
            tags.save(tag);
        } catch (Exception e) {
            throw new ApiException(HttpStatus.CONFLICT, "A tag with this name already
exists.");
        }
        return ResponseEntity.ok(new Dtos.TagResponse(tag.getId(), tag.getName()));
    }
}

```

What's happening: Parameters: numeric path id plus {name} body. Ownership is checked first — foreign IDs return 404 before validation even runs (so error precedence is 401 → 404 → 422 → 409). The rival lookup excludes the tag itself, permitting renames like 'work' → 'Work'. Save races collapse to 409. Returns 200 with the renamed tag.

Why it matters: The manager's Rename button maps here. Renaming automatically fixes lowerName via the model setter.

Byte 69: TagController.delete — removal plus contact cascade

Builds on: Bytes 6, 66

In plain terms: Deletes one owned tag and strips its ID from every contact that carried it, since MongoDB has no foreign keys.

The code:

```

@DeleteMapping("/{tags/{id}}")
public ResponseEntity<?> delete(@PathVariable long id, HttpServletRequest http) {
    User user = requireUser(http);
    Tag tag = tags.findByIdAndUserId(id, user.getId()).orElse(null);
    if (tag == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new
Dtos.ErrorResponse("Tag not found."));
    }
    tags.delete(tag);
    // remove tag from all contacts of this user
    var contacts = mongoTemplate.find(new
Query(Criteria.where("userId").is(user.getId()).and("tagIds").in(id)),
        com.phonebook.model.Contact.class);
    for (var c : contacts) {
        c.getTagIds().removeIf(t -> t == id);
        mongoTemplate.save(c);
    }
}

```

```

    }
    return ResponseEntity.ok(new Dtos.MessageResponse("Tag deleted successfully.));
}

```

What's happening: The cascade query finds only the caller's contacts embedding this tag (userId + tagIds \$in). removeSelf uses `t -> t == id` — reference comparison on Long objects, which works here only because unboxing applies to the `==` with a long... actually `t (Long) == id (long)` unboxes safely. Each touched contact is re-saved individually (no bulk), fine at tag scale. Returns 200 with a confirmation message.

Why it matters: Without this loop, deleted tags would linger as ghost IDs inside contacts and surface as missing pills. Gotcha: contacts of OTHER users are untouched even if they somehow referenced the id.

backend/src/main/java/com/phonebook/MongoIndexConfig.java

Byte 70: MongoIndexConfig — runner wiring

Builds on: Bytes 2, 6–10

In plain terms: A startup bean that ensures database indexes on every boot, idempotently.

The code:

```

package com.phonebook;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.index.Index;

@Configuration
public class MongoIndexConfig {
    @Bean
    public ApplicationRunner ensureIndexes(MongoTemplate mongoTemplate) {
        return args -> {

```

What's happening: `@Configuration` marks it as a bean factory; `@Bean` on `ensureIndexes` registers an `ApplicationRunner`, which Spring executes after the context is ready but before serving traffic. `MongoTemplate` gives raw index operations outside any repository. The lambda receives startup args (ignored here). Each `ensureIndex` call sits in its own try/catch so one existing/conflicting index never aborts the rest.

Why it matters: This runs before the first request on every boot — local, Docker, and test contexts alike.

Byte 71: MongoIndexConfig — the eight indexes

Builds on: Byte 71

In plain terms: Eight indexes encode the backend's uniqueness and performance contracts directly in the database.

The code:

```
try { mongoTemplate.indexOps("users").ensureIndex(new Index().on("username",
Sort.Direction.ASC).unique()); } catch (Exception ignored) {}
try { mongoTemplate.indexOps("users").ensureIndex(new Index().on("email",
Sort.Direction.ASC).unique()); } catch (Exception ignored) {}
try { mongoTemplate.indexOps("sessions").ensureIndex(new Index().on("tokenHash",
Sort.Direction.ASC).unique()); } catch (Exception ignored) {}
try { mongoTemplate.indexOps("contacts").ensureIndex(new Index().on("userId",
Sort.Direction.ASC)); } catch (Exception ignored) {}
try { mongoTemplate.indexOps("contacts").ensureIndex(new Index().on("phoneNumber",
Sort.Direction.ASC).unique()); } catch (Exception ignored) {}
try { mongoTemplate.indexOps("contacts").ensureIndex(new Index().on("email",
Sort.Direction.ASC).unique().sparse()); } catch (Exception ignored) {}
try { mongoTemplate.indexOps("tags").ensureIndex(new Index().on("userId",
Sort.Direction.ASC)); } catch (Exception ignored) {}
try { mongoTemplate.indexOps("tags").ensureIndex(new Index().on("userId",
Sort.Direction.ASC).on("lowerName", Sort.Direction.ASC).unique()); } catch (Exception
ignored) {}
```

What's happening: users.username/email unique back the 409s; sessions.tokenHash unique speeds every authenticated request. contacts.userId accelerates per-user listing; contacts.phoneNumber unique enforces global phone uniqueness; contacts.email unique SPARSE allows unlimited nulls while keeping real emails unique. tags.userId speeds listing; the compound (userId, lowerName) unique encodes per-user case-insensitive tag uniqueness. Empty catch blocks make reboots idempotent.

Why it matters: Application checks give friendly errors; these indexes give guarantees. If a 409 ever fires from a catch block instead of a pre-check, an index did its job.

backend/src/main/java/com/phonebook/PopulateConfig.java

Byte 72: PopulateConfig — trigger, math, and exits

Builds on: Bytes 2, 6, 16, 19

In plain terms: A dormant-unless-asked startup runner that tops the database up to 1000 contacts, then shuts the one-shot container down.

The code:

```

package com.phonebook;
// imports: Contact, ContactRepository, UserRepository, SequenceService, Instant,
collections, ThreadLocalRandom, ApplicationRunner/Bean/Configuration
private static final String[] FIRST = {"Aarav","Ananya","Arjun","Diya","Harsh", ...};
// 20 first names
private static final String[] LAST = {"Sharma","Verma","Patel","Iyer", ...}; //
20 last names
private static final String[] STREETS = {"MG Road","Linking Road","Park Street", ...};
// 10 streets
@Bean
public ApplicationRunner populateRunner(ContactRepository contacts, UserRepository
users, SequenceService sequences,
                                     org.springframework.context.ApplicationContext
context) {
    return args -> {
        boolean populate = false;
        for (String a : args.getSourceArgs()) {
            if ("populate".equalsIgnoreCase(a)) { populate = true; break; }
        }
        if (!populate) return; // normal boot: completely inert
        final int target = 1000;
        long existing = contacts.count();
        long needed = Math.max(target - existing, 0);
        if (needed == 0) {
            System.out.println("Database already contains " + existing + " contacts.");
            System.out.flush();
            int exitCode = org.springframework.boot.SpringApplication.exit(context, ()
-> 0);
            System.exit(exitCode);
            return;
        }
    }
}

```

What's happening: The word 'populate' in CLI args (docker compose run --rm backend populate) is the only trigger — ordinary boots return instantly. needed tops up to 1000 without touching existing rows. Both the already-full and the work-done paths terminate via SpringApplication.exit + System.exit(0), which is correct for the one-shot run container but would kill a long-lived server — hence it only runs when explicitly asked.

Why it matters: Demo pagination/search need volume; this is the sanctioned generator. Never pass 'populate' to the real server.

Byte 73: PopulateConfig — generating and saving the batch

Builds on: Byte 73

In plain terms: Random Indian-style names, unique US-format phones and emails, Mumbai addresses — all assigned to the first user.

The code:

```

var owner = users.findAll().stream().sorted((a, b) -> Long.compare(a.getId(),
b.getId())).findFirst().orElse(null);
    Set<String> phones = new
HashSet<>(contacts.findAll().stream().map(Contact::getPhoneNumber).toList());
    Set<String> emails = new HashSet<>(contacts.findAll().stream().filter(c ->
c.getEmail() != null).map(Contact::getEmail).toList());
    var random = ThreadLocalRandom.current();
    List<Contact> batch = new ArrayList<>();
    int guard = 0;
    while (batch.size() < needed && guard < needed * 20 + 500) {
        guard++;
        String phone = "+1" + (2000000000L + Math.abs(random.nextLong())) %
8000000000L);
        String email = ("user" + Math.abs(random.nextLong()) +
"@example.com").toLowerCase();
        if (phones.contains(phone) || emails.contains(email)) continue;
        String name = FIRST[random.nextInt(FIRST.length)] + " " +
LAST[random.nextInt(LAST.length)];
        String address = (100 + random.nextInt(900)) + " " +
STREETS[random.nextInt(STREETS.length)] + ", Mumbai";
        phones.add(phone);
        emails.add(email);
        Contact c = new Contact();
        c.setId(sequences.next("contacts"));
        c.setUserId(owner != null ? owner.getId() : null);
        c.setName(name);
        c.setPhoneNumber(phone);
        c.setEmail(email);
        c.setAddress(address);
        c.setCreatedAt(Instant.now());
        batch.add(c);
    }
    contacts.saveAll(batch);
    System.out.println("Added " + batch.size() + " contacts. Total contacts: " +
target + ".");
    System.out.flush();
    int exitCode = org.springframework.boot.SpringApplication.exit(context, () ->
0);
    System.exit(exitCode);

```

What's happening: Owner is the lowest-ID user (or null when no users exist — those become the orphans the first registration adopts). Preloaded phone/email sets plus in-loop additions prevent intra-batch and historical duplicates. The guard counter caps attempts so pathological collision streaks terminate. Phones like +1200... are syntactically valid per the phone rules. One saveAll persists the batch; the printed total is the target constant, not a recount.

Why it matters: Gotcha: generated contacts all belong to ONE user and carry no tags — multi-user and tag demos still need manual setup.

backend/src/test/resources/application.properties

Byte 74: Test config — isolated database

Builds on: Byte 4

In plain terms: The test classpath overrides one setting: use `phonebook_test` instead of `phonebook_db`.

The code:

```
server.port=8000
spring.data.mongodb.uri=${MONGODB_URI:mongodb://localhost:27017/phonebook_test}
spring.jackson.property-naming-strategy=SNAKE_CASE
spring.jackson.mapper.accept-case-insensitive-properties=true
spring.jackson.serialization.write-dates-as-timestamps=false
```

What's happening: Same Jackson behavior as main so tests exercise identical JSON. The `MONGODB_URI` default points at `phonebook_test`; Docker Compose overrides it with the container-network address. Because test resources shadow main resources on the test classpath, production data is unreachable during `mvn test`.

Why it matters: This file plus `ApiTestBase.cleanDatabase` is the two-layer test-isolation story.

backend/src/test/java/com/phonebook/ApiTestBase.java

Byte 75: ApiTestBase — harness wiring and database wipe

Builds on: Bytes 4, 75

In plain terms: The abstract parent of all test classes: boots the full app with a fake HTTP layer and empties every collection before each test.

The code:

```
package com.phonebook;
// imports: JsonNode/ObjectMapper, HashMap/Map/UUID, BeforeEach, Autowired,
// SpringBootTest, AutoConfigureMockMvc, MongoTemplate, MediaType,
MockHttpServletRequest, MockMvc/MvcResult ...
@SpringBootTest
@AutoConfigureMockMvc
public abstract class ApiTestBase {
    @Autowired protected MockMvc mvc;
    @Autowired protected ObjectMapper objectMapper;
    @Autowired protected MongoTemplate mongoTemplate;
    @BeforeEach
    void cleanDatabase() {
        for (String col : mongoTemplate.getCollectionNames()) {
            if (col.startsWith("system.")) continue;
        }
    }
}
```

```

        mongoTemplate.getCollection(col).deleteMany(new org.bson.Document());
    }
}

```

What's happening: @SpringBootTest starts the real application context (controllers, services, real MongoDB at phonebook_test); @AutoConfigureMockMvc provides the mvc fake-HTTP client — requests hit real controller/service/repository code with no port. cleanDatabase deletes every document in every non-system collection before EACH test, so tests are order-independent. Counters are wiped too, so IDs restart per test.

Why it matters: No test touches production data, and no test depends on another. That is the whole reliability story for the suite.

Byte 76: ApiTestBase — user and session helpers

Builds on: Byte 76

In plain terms: Three helpers: mint a unique user payload, register it (asserting 201), and extract the session cookie from a response.

The code:

```

protected Map<String, String> uniqueUser(String prefix) {
    String suffix = UUID.randomUUID().toString().replace("-", "").substring(0, 10);
    Map<String, String> user = new HashMap<>();
    user.put("username", prefix + "_" + suffix);
    user.put("email", prefix + "_" + suffix + "@example.com");
    user.put("password", "StrongPass123!");
    return user;
}

protected TestClient register(Map<String, String> user) throws Exception {
    MvcResult result = mvc.perform(post("/auth/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(user)))
        .andReturn();
    if (result.getResponse().getStatus() != 201) {
        throw new AssertionError("Register failed: " + result.getResponse().getStatus()
+ " " + result.getResponse().getContentAsString());
    }
    String cookie = sessionCookie(result.getResponse());
    return new TestClient(mvc, objectMapper, cookie);
}

protected static String sessionCookie(MockHttpServletRequest response) {
    String header = response.getHeader("Set-Cookie");
    if (header == null) return null;
    for (String part : header.split(";")) {
        part = part.trim();
        if (part.startsWith("phonebook_session=")) return
part.substring("phonebook_session=".length());
    }
}

```

```
    return null;
}
```

What's happening: uniqueUser's random suffix prevents cross-test collisions (registration is globally unique). register posts to the real /auth/register, fails loudly with status+body on non-201, and wraps the extracted cookie in a TestClient — so one line gives every test an authenticated actor. sessionCookie parses the Set-Cookie header by splitting on ';' and matching the exact cookie name.

Why it matters: Nearly every test opens with register(uniqueUser(...)). If registration breaks, all suites fail here first — a useful early signal.

Byte 77: ApiTestBase — contact payload and TestClient core

Builds on: Byte 77

In plain terms: A standard contact payload plus the cookie-carrying test client skeleton: construction and auth attachment.

The code:

```
protected Map<String, Object> contactPayload() {
    Map<String, Object> payload = new HashMap<>();
    payload.put("name", "Test Contact");
    payload.put("phone_number", "+14155550101");
    payload.put("email", "test-contact@example.com");
    payload.put("address", "123 Test Street");
    return payload;
}

protected static class TestClient {
    private final MockMvc mvc;
    private final ObjectMapper mapper;
    private String session;
    TestClient(MockMvc mvc, ObjectMapper mapper, String session) {
        this.mvc = mvc; this.mapper = mapper; this.session = session;
    }
    private org.springframework.test.web.servlet.request.MockHttpServletRequestBuilder
    withAuth(
        org.springframework.test.web.servlet.request.MockHttpServletRequestBuilder
    builder) {
        if (session != null) builder.cookie(new
jakarta.servlet.http.Cookie("phonebook_session", session));
        return builder;
    }
}
```

What's happening: contactPayload returns a fresh valid map per call (mutable copy per test, so overrides never leak between tests). TestClient stores the session token and withAuth attaches

it as a `phonebook_session` cookie to every request — mimicking a logged-in browser. Null session means anonymous (used implicitly only when helpers build fresh clients).

Why it matters: `contactPayload`'s fixed phone/email is why duplicate tests can reuse it verbatim: the second create with the same payload must 409.

Byte 78: `ApiTestBase` — `TestClient` HTTP verbs and JSON

Builds on: Byte 78

In plain terms: Five request methods plus session tracking and JSON parsing complete the fake browser.

The code:

```
public MvcResult post(String url, Object body) throws Exception {
    var builder = withAuth(...post(url).contentType(MediaType.APPLICATION_JSON));
    if (body != null) builder.content(mapper.writeValueAsString(body));
    MvcResult r = mvc.perform(builder).andReturn();
    updateSession(r);
    return r;
}

public MvcResult get(String url) throws Exception { ...withAuth(...get(url))...;
updateSession(r); return r; }

public MvcResult put(String url, Object body) throws Exception { ... }

public MvcResult delete(String url) throws Exception { ... }

public MvcResult multipart(String url, org.springframework.mock.web.MockMultipartFile
file) throws Exception {
    var builder = ...multipart(url).file(file);
    if (session != null) builder.cookie(new
jakarta.servlet.http.Cookie("phonebook_session", session));
    MvcResult r = mvc.perform(builder).andReturn();
    updateSession(r);
    return r;
}

private void updateSession(MvcResult r) {
    String header = r.getResponse().getHeader("Set-Cookie");
    if (header != null && header.contains("phonebook_session=")) {
        String c = ApiTestBase.sessionCookie(r.getResponse());
        if (c != null && !c.isEmpty()) this.session = c;
        else if (header.contains("Max-Age=0")) this.session = null;
    }
}

public JsonNode json(MvcResult r) throws Exception {
    return mapper.readTree(r.getResponse().getContentAsString());
}
```

What's happening: Each verb serializes the body (null body sends none), performs, refreshes the session, and returns the raw result for status+body assertions. `multipart` attaches the file part for CSV import with the cookie added manually (`multipart` builders skip `withAuth`'s chain).

updateSession adopts refreshed cookies (login rotation) and nulls the session on Max-Age=0 (logout) — so client.post('/auth/logout') genuinely de-authenticates later calls. json parses bodies for field assertions.

Why it matters: Multi-step flows (register → create → search → logout → 401) work because the client behaves like a cookie jar.

backend/src/test/java/com/phonebook/AuthTests.java

Byte 79: AuthTests — root and successful registration

Builds on: Bytes 27, 45, 76–79

In plain terms: The suite opens with the health check and the happy-path registration: 201 plus a session cookie.

The code:

```
package com.phonebook;
// imports: JsonNode, HashMap/Map, Test, MediaType, junit assertions, MockMvc
builders/matchers
public class AuthTests extends ApiTestBase {
    @Test
    void rootEndpointReturnsRunningMessage() throws Exception {
        var result = mvc.perform(get("/")).andReturn();
        assertEquals(200, result.getResponse().getStatus());
        JsonNode body =
objectMapper.readTree(result.getResponse().getContentAsString());
        assertEquals("Phonebook API is running", body.get("message").asText());
    }
    @Test
    void registerValidUserCreatesAccountAndSessionCookie() throws Exception {
        var user = uniqueUser("testuser");
        var result = mvc.perform(post("/auth/register")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(user))).andReturn();
        assertEquals(201, result.getResponse().getStatus());
        JsonNode body =
objectMapper.readTree(result.getResponse().getContentAsString());
        assertEquals(user.get("email"), body.get("email").asText());
        String setCookie = result.getResponse().getHeader("Set-Cookie");
        assertNotNull(setCookie);
        assertTrue(setCookie.startsWith("phonebook_session="));
    }
}
```

What's happening: The root test uses raw mvc (no client) and parses JSON by hand — the only test touching RootController. The registration test asserts three things: status 201, echoed

email, and a Set-Cookie header starting with the exact cookie name — proving register_issueSession wiring end to end.

Why it matters: If the cookie assertion fails while 201 passes, session issuance broke but user creation survived — the split pinpoints the layer.

Byte 80: AuthTests — duplicates and invalid input

Builds on: Byte 80

In plain terms: Duplicate usernames/emails must 409; short usernames, bad emails, and short passwords must 422 together.

The code:

```
@Test
void registerDuplicateUsernameOrEmailReturnsConflict() throws Exception {
    var user = uniqueUser("testuser");
    mvc.perform(post("/auth/register").contentType(MediaType.APPLICATION_JSON)

        .content(objectMapper.writeValueAsString(user))).andExpect(status().isCreated());
    Map<String, String> dupUser = new HashMap<>(user);
    dupUser.put("email", "different@example.com");
    mvc.perform(post("/auth/register").contentType(MediaType.APPLICATION_JSON)

        .content(objectMapper.writeValueAsString(dupUser))).andExpect(status().isConflict());
    Map<String, String> dupEmail = new HashMap<>(user);
    dupEmail.put("username", "different_username");
    mvc.perform(post("/auth/register").contentType(MediaType.APPLICATION_JSON)

        .content(objectMapper.writeValueAsString(dupEmail))).andExpect(status().isConflict());
}
@Test
void registerInvalidDataReturnsUnprocessable() throws Exception {
    Map<String, String> bad = Map.of("username", "ab", "email", "not-an-email",
    "password", "short");
    mvc.perform(post("/auth/register").contentType(MediaType.APPLICATION_JSON)

        .content(objectMapper.writeValueAsString(bad))).andExpect(status().isUnprocessableEntity());
}
```

What's happening: Each duplicate case changes exactly one field, isolating username vs email conflicts. The invalid payload violates all three rules at once ('ab', 'not-an-email', 'short') and expects a single 422 — proving the validator collects rather than short-circuits. Uses MockMvc's andExpect matcher style instead of manual assertEquals.

Why it matters: These lock the 409/422 contract the register form depends on for per-field vs conflict messaging.

Byte 81: AuthTests — login both ways and bad credentials

Builds on: Bytes 49, 79

In plain terms: Login works with username or email after a logout, and unknown users get a flat 401.

The code:

```
@Test
void loginByUsernameAndEmailSucceeds() throws Exception {
    var user = uniqueUser("testuser");
    TestClient client = register(user);
    client.post("/auth/logout", null);
    for (String identifier : new String[]{user.get("username"), user.get("email")}) {
        var r = client.post("/auth/login", Map.of("identifier", identifier, "password",
user.get("password")));
        assertEquals(200, r.getResponse().getStatus());
        JsonNode body = client.json(r);
        assertEquals(user.get("username"), body.get("username").asText());
        client.post("/auth/logout", null);
    }
}

@Test
void loginInvalidCredentialsReturnsUnauthorized() throws Exception {
    var r = mvc.perform(post("/auth/login").contentType(MediaType.APPLICATION_JSON)
        .content("{\"identifier\":\"missing-user\",\"password\":\"wrong-
password\"}")).andReturn();
    assertEquals(401, r.getResponse().getStatus());
}
```

What's happening: The loop logs out first (so the test proves login, not lingering registration session), then logs in twice — once per identifier — logging out between rounds, which also exercises updateSession's cookie refresh. The invalid test uses a raw JSON string and anonymous mvc, asserting only the 401 (never distinguishing 'unknown user' from 'wrong password').

Why it matters: Passing means the identifier-or-email query, password verification, and session rotation all work in sequence.

Byte 82: AuthTests — me, logout, script compatibility

Builds on: Bytes 32, 50, 79

In plain terms: Anonymous /auth/me is 401 while authed is 200; logout invalidates; old Python-era password hashes still verify.

The code:

```
@Test
void currentUserRequiresAuthentication() throws Exception {
    var anon = mvc.perform(get("/auth/me")).andReturn();
```

```

    assertEquals(401, anon.getResponse().getStatus());
    var user = uniqueUser("testuser");
    TestClient client = register(user);
    var me = client.get("/auth/me");
    assertEquals(200, me.getResponse().getStatus());
    assertEquals(user.get("email"), client.json(me).get("email").asText());
}
@Test
void logoutInvalidatesSession() throws Exception {
    var user = uniqueUser("testuser");
    TestClient client = register(user);
    assertEquals(200, client.get("/auth/me").getResponse().getStatus());
    assertEquals(200, client.post("/auth/logout", null).getResponse().getStatus());
    assertEquals(401, client.get("/auth/me").getResponse().getStatus());
}
@Test
void passwordHashIsCompatibleWithPythonScrypt() {
    var security = new com.phonebook.service.SecurityService();
    String pythonHash = "scrypt$AAECAwQFBgcICQoLDA0ODw==$-
TWpMCXcNA8tK2qNm9ZnzhACHcOUIsMX2JcTb1KjZr-E0YSC3CSTf0Nt0CQ-
ITS3ZVYTuhmr2CFnAN_6yGWWNA==";
    assertTrue(security.verifyPassword("StrongPass123!", pythonHash));
    assertFalse(security.verifyPassword("wrong-password", pythonHash));
    assertTrue(security.verifyPassword("round-trip-password",
security.hashPassword("round-trip-password")));
}

```

What's happening: The me test contrasts anonymous vs authenticated in one method. The logout test's final 401 passes through updateSession's Max-Age=0 nulling — proving client and server agree the session died. The scrypt test needs no Spring context: it pins cross-era hash compatibility with a real Python-generated vector plus a Java round-trip.

Why it matters: Together: session restore, session death, and password-hash portability are locked.

backend/src/test/java/com/phonebook/ContactTests.java

Byte 83: ContactTests — auth gate and create/read/list

Builds on: Bytes 51–54, 60, 76–79

In plain terms: Anonymous contact calls all 401; an authed create→read→list round-trip proves the core lifecycle and envelope.

The code:

```

package com.phonebook;
// imports: JsonNode, HashMap/Map, Test, junit assertions
public class ContactTests extends ApiTestBase {

```

```

@Test
void protectedContactEndpointsRequireAuthentication() throws Exception {
    assertEquals(401,
mvc.perform(...get("/contacts/")).andReturn().getResponse().getStatus());
    var payload = contactPayload();
    assertEquals(401,
mvc.perform(...post("/contacts/")...content(...writeValueAsString(payload))).andReturn().getResponse().getStatus());
    assertEquals(401,
mvc.perform(...get("/contacts/1")).andReturn().getResponse().getStatus());
}
@Test
void createAndGetAndListContacts() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    var created = client.post("/contacts/", contactPayload());
    assertEquals(201, created.getResponse().getStatus());
    JsonNode contact = client.json(created);
    var single = client.get("/contacts/" + contact.get("id").asLong());
    var listing = client.get("/contacts/?page=1&limit=10");
    assertEquals(200, single.getResponse().getStatus());
    assertEquals("+14155550101", client.json(single).get("phone_number").asText());
    assertEquals(200, listing.getResponse().getStatus());
    assertEquals(1, client.json(listing).get("total").asInt());
    assertEquals(contact.get("id").asLong(),
client.json(listing).get("items").get(0).get("id").asLong());
}

```

What's happening: The gate test covers list, create, and single-read anonymously. The lifecycle test chains three calls on one client: create (201), fetch by echoed id (phone matches verbatim), list (total=1 with the same id) — proving IDs, snake_case serialization, and the envelope agree.

Why it matters: This is the minimum 'contacts work' proof; everything after refines it.

Byte 84: ContactTests — invalid data and duplicates

Builds on: Bytes 55, 56

In plain terms: Bad phones 422; reused phone or email 409s — each duplicate dimension isolated.

The code:

```

@Test
void createInvalidContactDataReturnsUnprocessable() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    Map<String, Object> invalid = contactPayload();
    invalid.put("phone_number", "not-a-phone");
    assertEquals(422, client.post("/contacts/", invalid).getResponse().getStatus());
}
@Test
void duplicatePhoneAndEmailAreRejected() throws Exception {

```

```

    TestClient client = register(uniqueUser("testuser"));
    assertEquals(201, client.post("/contacts/",
contactPayload()).getResponse().getStatus());
    Map<String, Object> dupPhone = contactPayload();
    dupPhone.put("email", "other@example.com");
    assertEquals(409, client.post("/contacts/", dupPhone).getResponse().getStatus());
    Map<String, Object> dupEmail = contactPayload();
    dupEmail.put("phone_number", "+14155550102");
    assertEquals(409, client.post("/contacts/", dupEmail).getResponse().getStatus());
}

```

What's happening: contactPayload's fixed phone/email makes duplication trivial: change only email to isolate the phone conflict and vice versa. 'not-a-phone' fails the phone regex path (422, not 409 — validation precedes duplicate checks, matching controller order).

Why it matters: Locks the 422-before-409 precedence and the global-uniqueness rule in two dimensions.

Byte 85: ContactTests — update and delete

Builds on: Bytes 61–63

In plain terms: Partial update changes only sent fields; delete removes and subsequent reads/deletes 404.

The code:

```

@Test
void updateContactChangesProvidedFields() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    JsonNode created = client.json(client.post("/contacts/", contactPayload()));
    var response = client.put("/contacts/" + created.get("id").asLong(),
        Map.of("name", "Updated Contact", "address", "456 Updated Avenue"));
    assertEquals(200, response.getResponse().getStatus());
    JsonNode body = client.json(response);
    assertEquals("Updated Contact", body.get("name").asText());
    assertEquals("456 Updated Avenue", body.get("address").asText());
}

@Test
void deleteContactAndMissingContact() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    JsonNode created = client.json(client.post("/contacts/", contactPayload()));
    long id = created.get("id").asLong();
    assertEquals(200, client.delete("/contacts/" + id).getResponse().getStatus());
    assertEquals(404, client.get("/contacts/" + id).getResponse().getStatus());
    assertEquals(404, client.delete("/contacts/999999").getResponse().getStatus());
}

```

What's happening: The update sends only name+address (phone/email/tags untouched — implicitly the partial-mode contract) and asserts both changed values echo back. The delete test proves the full death arc: 200 delete → 404 read → 404 re-delete, plus 404 for a never-existent id. Map.of builds the immutable PUT bodies.

Why it matters: Passing means partial updates preserve unmentioned fields and deletion is total and idempotent-safe (second delete 404s, never 500s).

Byte 86: ContactTests — search, pagination, isolation

Builds on: Bytes 53, 54, 60

In plain terms: Twelve contacts prove page math and search precision; a second user sees nothing of the first.

The code:

```
@Test
void searchAndPagination() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    for (int i = 0; i < 12; i++) {
        char letter = (char) ('A' + i);
        Map<String, Object> p = new HashMap<>();
        p.put("name", "Search Person " + letter);
        p.put("phone_number", String.format("+1415555%04d", 1000 + i));
        p.put("email", "search" + i + "@example.com");
        p.put("address", "123 Search Street");
        assertEquals(201, client.post("/contacts/", p).getResponse().getStatus());
    }
    JsonNode page = client.json(client.get("/contacts/?page=2&limit=10"));
    JsonNode search = client.json(client.get("/contacts/?page=1&limit=10&search=Search
Person A"));
    assertEquals(2, page.get("page").asInt());
    assertEquals(10, page.get("limit").asInt());
    assertEquals(12, page.get("total").asInt());
    assertEquals(2, page.get("items").size());
    assertEquals(1, search.get("total").asInt());
    assertEquals("Search Person A", search.get("items").get(0).get("name").asText());
}

@Test
void usersCannotAccessEachOthersContacts() throws Exception {
    TestClient first = register(uniqueUser("first"));
    JsonNode created = first.json(first.post("/contacts/", contactPayload()));
    first.post("/auth/logout", null);
    TestClient second = register(new java.util.HashMap<>(Map.of("username", "second-
user", "email", "second-user@example.com", "password", "StrongPass123!")));
    JsonNode listing = second.json(second.get("/contacts/?page=1&limit=10"));
    assertEquals(404, second.get("/contacts/" +
created.get("id").asLong()).getResponse().getStatus());
    assertEquals(404, second.put("/contacts/" + created.get("id").asLong(),
Map.of("name", "Hijacked Contact")).getResponse().getStatus());
}
```

```

    assertEquals(404, second.delete("/contacts/" +
created.get("id").asLong()).getResponse().getStatus());
    assertEquals(0, listing.get("total").asInt());
}

```

What's happening: Twelve 'Search Person A–L' contacts with sequenced phones/emails: page 2 of 10 must hold exactly 2 items with total 12 (ceiling-division proof), and search matches exactly one. The isolation test logs the first user out, registers a fixed second user (DB is wiped per test, so fixed names are safe), and asserts 404 on read/update/delete of the foreign id plus an empty own listing.

Why it matters: User isolation is proven in four operations at once — the security test that matters most for a multi-tenant phonebook.

backend/src/test/java/com/phonebook/TagTests.java

Byte 87: TagTests — helpers and auth gate

Builds on: Bytes 66–70, 76–79

In plain terms: Two private helpers (tag + tagged-contact creation) plus proof that anonymous tag reads/writes 401.

The code:

```

package com.phonebook;
// imports: JsonNode, List/Map, Test, junit assertions
public class TagTests extends ApiTestBase {
    private long createTag(TestClient client, String name) throws Exception {
        var r = client.post("/tags/", Map.of("name", name));
        assertEquals(201, r.getResponse().getStatus());
        return client.json(r).get("id").asLong();
    }
    private long createTaggedContact(TestClient client, String name, String phone,
String email, List<Long> tagIds) throws Exception {
        var r = client.post("/contacts/", Map.of("name", name, "phone_number", phone,
"email", email,
        "address", "123 Test Street", "tag_ids", tagIds));
        assertEquals(201, r.getResponse().getStatus());
        return client.json(r).get("id").asLong();
    }
    @Test
    void protectedTagEndpointsRequireAuthentication() throws Exception {
        assertEquals(401,
mvc.perform(...get("/tags/")).andReturn().getResponse().getStatus());
        assertEquals(401,
mvc.perform(...post("/tags/")...content("{\"name\":\"Work\"}")).andReturn().getResponse().g
etStatus());
    }
}

```

What's happening: createTag returns the numeric id for later filter/update/delete calls; createTaggedContact wires tag_ids at creation and returns the contact id. Both assert 201 internally so failures point at setup, not assertions. The gate test uses raw JSON for the POST body and covers list + create anonymously.

Why it matters: Every later tag test is three lines of setup thanks to these helpers — the file's readability trick.

Byte 88: TagTests — lifecycle, duplicates, invalid names

Builds on: Bytes 67–69

In plain terms: Trim-on-create, list, rename, delete, missing-tag 404; case-insensitive duplicates 409; bad names 422.

The code:

```
@Test
void createListUpdateAndDeleteTags() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    var created = client.post("/tags/", Map.of("name", " Family "));
    assertEquals(201, created.getResponse().getStatus());
    JsonNode tag = client.json(created);
    assertEquals("Family", tag.get("name").asText());
    JsonNode listing = client.json(client.get("/tags/"));
    assertEquals(1, listing.size());
    assertEquals(tag.get("id").asLong(), listing.get(0).get("id").asLong());
    var updated = client.put("/tags/" + tag.get("id").asLong(), Map.of("name",
"Personal"));
    assertEquals(200, updated.getResponse().getStatus());
    assertEquals("Personal", client.json(updated).get("name").asText());
    assertEquals(200, client.delete("/tags/" +
tag.get("id").asLong()).getResponse().getStatus());
    assertEquals(0, client.json(client.get("/tags/")).size());
    assertEquals(404, client.delete("/tags/999999").getResponse().getStatus());
}

@Test
void duplicateAndInvalidTagNamesAreRejected() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    assertEquals(201, client.post("/tags/", Map.of("name",
"Work")).getResponse().getStatus());
    assertEquals(409, client.post("/tags/", Map.of("name",
"work")).getResponse().getStatus());
    assertEquals(422, client.post("/tags/", Map.of("name",
"")).getResponse().getStatus());
    assertEquals(422, client.post("/tags/", Map.of("name",
"bad,name")).getResponse().getStatus());
}
```

What's happening: 'Family' echoing back as 'Family' proves controller trimming; listing asserts array shape (bare JSON array, not an envelope). 'work' vs 'Work' proves lowerName uniqueness; empty string and comma names prove the validator pattern. Unknown-id delete 404s.

Why it matters: The full tag lifecycle plus both rejection dimensions in two methods.

Byte 89: TagTests — assignment, clearing, tag-preserving updates

Builds on: Bytes 61–63, 68

In plain terms: Contacts take multiple tags, narrow to one, clear to zero — and a tag-less update leaves tags untouched.

The code:

```
@Test
void assignMultipleTagsToContactsAndClearThem() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    long work = createTag(client, "Work");
    long family = createTag(client, "Family");
    var created = client.post("/contacts/", Map.of("name", "Tagged Contact",
"phone_number", "+14155550101",
    "email", "tagged@example.com", "address", "123 Test Street", "tag_ids",
List.of(work, family)));
    assertEquals(201, created.getResponse().getStatus());
    assertEquals(2, client.json(created).get("tags").size());
    long contactId = client.json(created).get("id").asLong();
    var updated = client.put("/contacts/" + contactId, Map.of("tag_ids",
List.of(work)));
    assertEquals(200, updated.getResponse().getStatus());
    JsonNode updatedBody = client.json(updated);
    assertEquals(1, updatedBody.get("tags").size());
    assertEquals("Work", updatedBody.get("tags").get(0).get("name").asText());
    var cleared = client.put("/contacts/" + contactId, Map.of("tag_ids", List.of()));
    assertEquals(0, client.json(cleared).get("tags").size());
}

@Test
void updateWithoutTagIdsLeavesTagsUnchanged() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    long work = createTag(client, "Work");
    var createdRes = client.post("/contacts/", Map.of("name", "Tagged Contact",
"phone_number", "+14155550101",
    "email", "tagged@example.com", "address", "123 Test Street", "tag_ids",
List.of(work)));
    JsonNode created = client.json(createdRes);
    var updated = client.put("/contacts/" + created.get("id").asLong(), Map.of("name",
"Tagged Contact Updated"));
    assertEquals(200, updated.getResponse().getStatus());
    JsonNode body = client.json(updated);
    assertEquals("Tagged Contact Updated", body.get("name").asText());
    assertEquals(1, body.get("tags").size());
    assertEquals(work, body.get("tags").get(0).get("id").asLong());
}
```

```
}
```

What's happening: The first test walks $2 \rightarrow 1 \rightarrow 0$ tags across three PUTs, asserting resolved tag objects (not IDs) including the surviving name. The second proves the absent-key contract: a PUT without tag_ids keeps the tag (id-equality asserted). Together they pin the null/absent-vs-empty distinction from Byte 61.

Why it matters: This pair is the executable spec for the update semantics the edit modal relies on.

Byte 90: TagTests — tag filtering, search combo, tag isolation

Builds on: Bytes 53, 54, 62, 70

In plain terms: \$all filtering, search+tag combination, tag pagination — then proof that users cannot touch each other's tags.

The code:

```
@Test
void filterContactsByTagsAndCombineWithTextSearch() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    long work = createTag(client, "Work");
    long family = createTag(client, "Family");
    createTaggedContact(client, "Alice Work", "+14155551001", "alice-work@example.com",
        List.of(work));
    createTaggedContact(client, "Alice Family", "+14155551002", "alice-
family@example.com", List.of(family));
    createTaggedContact(client, "Bob Both", "+14155551003", "bob-both@example.com",
        List.of(work, family));
    JsonNode byWork = client.json(client.get("/contacts/?page=1&limit=10&tag_ids=" +
work));
    JsonNode byBoth = client.json(client.get("/contacts/?page=1&limit=10&tag_ids=" +
work + "&tag_ids=" + family));
    JsonNode combined =
client.json(client.get("/contacts/?page=1&limit=10&search=Alice&tag_ids=" + work));
    JsonNode paged = client.json(client.get("/contacts/?page=1&limit=1&tag_ids=" +
work));
    assertEquals(2, byWork.get("total").asInt());
    assertEquals(1, byBoth.get("total").asInt());
    assertEquals("Bob Both", byBoth.get("items").get(0).get("name").asText());
    assertEquals(1, combined.get("total").asInt());
    assertEquals("Alice Work", combined.get("items").get(0).get("name").asText());
    assertEquals(2, paged.get("total").asInt());
    assertEquals(1, paged.get("items").size());
    assertEquals(2, paged.get("total_pages").asInt());
}

@Test
void usersCannotAccessEachOthersTagsOrAssignThem() throws Exception {
    TestClient first = register(uniqueUser("first"));
    long firstTag = createTag(first, "Secret");
```

```

    long firstContact = createTaggedContact(first, "First Person", "+14155552001",
"first-person@example.com", List.of(firstTag));
    first.post("/auth/logout", null);
    TestClient second = register(uniqueUser("second"));
    assertEquals(0, second.json(second.get("/tags/")).size());
    assertEquals(404, second.put("/tags/" + firstTag, Map.of("name",
"Stolen")).getResponse().getStatus());
    assertEquals(404, second.delete("/tags/" + firstTag).getResponse().getStatus());
    long ownTag = createTag(second, "Mine");
    assertEquals(400, second.post("/contacts/", Map.of("name", "Second Person",
"phone_number", "+14155552002",
    "email", "second-person@example.com", "address", "123 Test Street",
"tag_ids", List.of(firstTag))).getResponse().getStatus());
    long ownContact = createTaggedContact(second, "Second Person", "+14155552002",
"second-person@example.com", List.of(ownTag));
    assertEquals(400, second.put("/contacts/" + ownContact, Map.of("tag_ids",
List.of(firstTag))).getResponse().getStatus());
    assertEquals(0, second.json(second.get("/contacts/?page=1&limit=10&tag_ids=" +
firstTag)).get("total").asInt());
    assertEquals(404, second.get("/contacts/" +
firstContact).getResponse().getStatus());
}

```

What's happening: Three contacts (Work-only, Family-only, both) prove \$all: work matches 2, work+family matches only Bob Both; Alice+Work narrows to one; limit=1 over 2 matches yields total_pages=2. The isolation test proves six denials: empty foreign tag list, 404 rename/delete of a foreign tag, 400 attaching a foreign tag on create AND update, zero foreign-tag filter hits, and 404 reading the foreign contact — pinning the 404-vs-400 distinction.

Why it matters: Filtering semantics and the tag-security model, fully specified in two methods.

backend/src/test/java/com/phonebook/ImportExportTests.java

Byte 91: ImportExportTests — CSV helper and auth gate

Builds on: Bytes 57–59, 76–79

In plain terms: A one-line CSV file factory plus proof that export and import both require login.

The code:

```

package com.phonebook;
// imports: JsonNode, StandardCharsets, HashSet/Map/Set, Test, MockMultipartFile, junit
assertions
public class ImportExportTests extends ApiTestBase {
    private MockMultipartFile csvFile(String content, String filename) {
        return new MockMultipartFile("file", filename, "text/csv",
content.getBytes(StandardCharsets.UTF_8));
    }
}

```

```

@Test
void exportAndImportRequireAuthentication() throws Exception {
    assertEquals(401,
mvc.perform(...get("/contacts/export")).andReturn().getResponse().getStatus());
    MockMultipartFile file = csvFile("name,phone_number\nA,+14155550101\n",
"contacts.csv");
    assertEquals(401,
mvc.perform(...multipart("/contacts/import").file(file)).andReturn().getResponse().getSta
tus());
}

```

What's happening: csvFile builds the multipart 'file' part with explicit UTF-8 bytes and a chosen filename — the filename's extension drives the .csv gate, so tests control it directly. The gate test hits both endpoints anonymously: GET export and multipart POST import each 401. Note the import 401 fires before any CSV parsing (auth precedes file checks).

Why it matters: Every later import test is one csvFile(...) call — filename and content are the two test dials.

Byte 92: ImportExportTests — export scoping and happy import

Builds on: Bytes 57, 59

In plain terms: Export contains only the caller's contacts; a two-row CSV imports fully and becomes searchable.

The code:

```

@Test
void exportReturnsOnlyAuthenticatedUsersContacts() throws Exception {
    TestClient first = register(uniqueUser("first"));
    JsonNode created = first.json(first.post("/contacts/", contactPayload()));
    first.post("/auth/logout", null);
    TestClient second = register(Map.of("username", "export_other_user", "email",
"export_other_user@example.com", "password", "StrongPass123!"));
    Map<String, Object> other = contactPayload();
    other.put("name", "Other User Contact");
    other.put("phone_number", "+14155550102");
    other.put("email", "other@example.com");
    assertEquals(201, second.post("/contacts/", other).getResponse().getStatus());
    var response = second.get("/contacts/export");
    assertEquals(200, response.getResponse().getStatus());
    String contentType = response.getResponse().getContentType();
    assertTrue(contentType != null && contentType.startsWith("text/csv"));
    String csv = response.getResponse().getContentAsString();
    assertTrue(csv.contains("Name,Phone number,Email,Address"));
    assertTrue(csv.contains("Other User Contact"));
    assertTrue(csv.contains("+14155550102"));
    assertFalse(csv.contains(created.get("name").asText()));
}
@Test

```

```

void importValidCsvPersistsContacts() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    String content = "name,phone_number,email,address\n"
        + "Imported Alpha,+14155550121,alpha@example.com,123 Import Street\n"
        + "Imported Beta,+14155550122,beta@example.com,456 Import Avenue\n";
    var response = client.multipart("/contacts/import", csvFile(content,
"contacts.csv"));
    assertEquals(200, response.getResponse().getStatus());
    JsonNode summary = client.json(response);
    JsonNode contacts =
client.json(client.get("/contacts/?page=1&limit=10&search=Imported"));
    assertEquals(2, summary.get("total_rows").asInt());
    assertEquals(2, summary.get("imported").asInt());
    assertEquals(2, contacts.get("total").asInt());
    Set<String> emails = new HashSet<>();
    contacts.get("items").forEach(item -> emails.add(item.get("email").asText()));
    assertEquals(Set.of("alpha@example.com", "beta@example.com"), emails);
}

```

What's happening: The export test uses a fixed-credential second user (safe: DB wiped per test), then asserts header line, content type, own contact present, and first user's contact name absent. The import test asserts summary counts AND searchability, collecting result emails into a set for exact comparison — proving rows persist, not just count.

Why it matters: Export isolation and import persistence — the two promises of the CSV feature.

Byte 93: ImportExportTests — foreign columns, Excel phones, invalid rows

Builds on: Bytes 25, 40, 59

In plain terms: First/Last/Phone/City/Country columns compose correctly; Excel formula phones unwrap; scientific emails/names fail with exact per-field tallies.

The code:

```

@Test
void importExternalColumnsAndExcelPhoneText() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    String content = "First Name,Last Name,Email,Phone,City,Country\n"
        + "External,Contact,external@example.com,="+14155550131\","Mumbai,India\n";
    var response = client.multipart("/contacts/import", csvFile(content,
"contacts.csv"));
    assertEquals(200, response.getResponse().getStatus());
    JsonNode summary = client.json(response);
    JsonNode contacts =
client.json(client.get("/contacts/?page=1&limit=10&search=External"));
    assertEquals(1, summary.get("imported").asInt());
    assertEquals("+14155550131",
contacts.get("items").get(0).get("phone_number").asText());
    assertEquals("Mumbai India", contacts.get("items").get(0).get("address").asText());
}

```

```

@Test
void importInvalidRowsAndScientificNotation() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    String content = "name,phone_number,email,address\n"
        + "Invalid Phone,9.11235E+11,valid@example.com,123 Invalid Street\n"
        + "Invalid Email,+14155550132,not-an-email,123 Invalid Street\n"
        + ",+14155550133,missing@example.com,123 Invalid Street\n";
    var response = client.multipart("/contacts/import", csvFile(content,
"contacts.csv"));
    assertEquals(200, response.getResponse().getStatus());
    JsonNode summary = client.json(response);
    assertEquals(3, summary.get("total_rows").asInt());
    assertEquals(0, summary.get("imported").asInt());
    assertEquals(3, summary.get("invalid_rows").asInt());
    assertEquals(1, summary.get("invalid_phone_numbers").asInt());
    assertEquals(1, summary.get("invalid_emails").asInt());
    assertEquals(1, summary.get("invalid_names").asInt());
}

```

What's happening: The external-column test is the alias/normalization spec in one row: name composed from First+Last, Phone alias, address from City+Country joined with a space, Excel = "..." unwrapped to the raw number. The invalid test crafts one failure per field (scientific phone, bad email, missing name) and asserts each counter hits exactly 1 with zero imports — even failures return HTTP 200 with a summary, not an error status.

Why it matters: These two tests are why the importer can claim broad compatibility AND precise accounting.

Byte 94: ImportExportTests — duplicates, headers, file rejections

Builds on: Bytes 58, 59

In plain terms: Re-imports skip everything; missing name/phone columns 400 with a name hint; wrong extension and empty files 400.

The code:

```

@Test
void importDuplicatesAreSkipped() throws Exception {
    TestClient client = register(uniqueUser("testuser"));
    String content = "name,phone_number,email,address\n"
        + "First Import,+14155550141,first@example.com,123 Duplicate Street\n"
        + "Duplicate Phone,+14155550142,first@example.com,789 Duplicate Street\n";
    JsonNode first = client.json(client.multipart("/contacts/import", csvFile(content,
"contacts.csv")));
    JsonNode second = client.json(client.multipart("/contacts/import", csvFile(content,
"contacts.csv")));
    assertEquals(1, first.get("imported").asInt());
    assertEquals(2, first.get("skipped_duplicates").asInt());
    assertEquals(0, second.get("imported").asInt());
}

```

```

        assertEquals(3, second.get("skipped_duplicates").asInt());
    }
    @Test
    void importRejectsMissingHeaders() throws Exception {
        TestClient client = register(uniqueUser("testuser"));
        var response = client.multipart("/contacts/import",
        csvFile("email,address\na@example.com,123 Street\n", "contacts.csv"));
        assertEquals(400, response.getResponse().getStatus());
        JsonNode body = client.json(response);
        assertTrue(body.get("detail").asText().toLowerCase().contains("name"));
    }
    @Test
    void importRejectsNonCsvAndEmptyFiles() throws Exception {
        TestClient client = register(uniqueUser("testuser"));
        assertEquals(400, client.multipart("/contacts/import",
        csvFile("name,phone\nTest,+14155550151\n", "contacts.txt")).getResponse().getStatus());
        assertEquals(400, client.multipart("/contacts/import", csvFile("",
        "empty.csv")).getResponse().getStatus());
    }
}

```

What's happening: Note the fixture's second row reuses the first row's EMAIL (not phone) — 'Duplicate Phone' is a misnomer; both trailing rows duplicate on email, and first-import math (1 imported + 2 skipped) proves intra-file dedup via the growing seen-sets. The full re-import then skips all 3. Missing headers assert the 400 message names the problem ('name'). The .txt upload fails on extension despite valid content; the empty file fails on emptiness.

Why it matters: Idempotency, header gating, and file gating — the suite's closing edge cases. Total backend coverage: 32 tests, all green.